

Titreşim İzleme Node'u

CAN tabanlı titreşim izleme · Kablosuz gateway · Uzaktan firmware güncelleme

Proje Planı ve Uygulama Rehberi

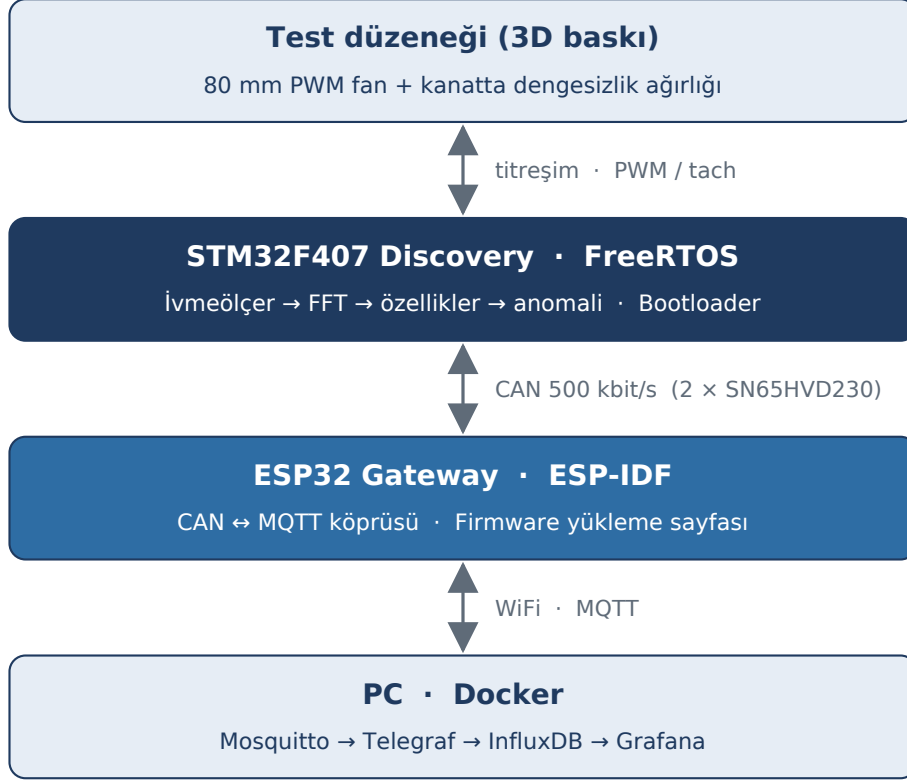
Süre	4 hafta. Hafta 3 sonunda proje yayınlanabilir durumda; Hafta 4 bonus.
Ana donanım	STM32F407 Discovery (MB997D), ESP32, 2 × SN65HVD230 CAN transceiver, Arctic P8 PWM fan, lojik analizör
Ek harcama	1.974,82 TL — sipariş verildi (Bölüm 3)
Çalışma şekli	Planlama bu dokümanla; uygulama Claude Code ile, doğrulama senin elinle
Hedef	Savunma / endüstriyel gömülü yazılım pozisyonları için uçtan uca, bitmiş bir portfolyo projesi

Bu doküman nasıl kullanılır

Bölüm 1-2	Projenin ne olduğu ve başlamadan önce kontrol etmen gerekenler.
Bölüm 3-4	Malzeme durumu (satın alınanlar, evde olanlar, 3D baskı) ve kurulacak yazılımlar.
Bölüm 5-6	Kablolama (pin tabloları) ve 3D baskı parçaları.
Bölüm 7	Teknik tasarım. Bu bölüm repo'daki SPEC.md dosyasının kaynağıdır; Claude Code bunu sözleşme olarak kullanır.
Bölüm 8-9	Repo yapısı ve Claude Code ile çalışma yöntemi (CLAUDE.md şablonu dahil).
Bölüm 10	Haftalık iş listesi, “bitti” kriterleri ve konu kartları : her görevin arkasındaki kavram, kaynaklar ve test soruları.
Bölüm 11-12	Sık yapılan hatalar ve teslim: README, demo videosu, CV satırı.
Ek A-B	Test sorularının cevap anahtarı ve tüm kaynakların listesi.

1. Proje Özeti

STM32F407 Discovery kartı, üzerindeki ivmeölçerle 3D baskı bir düzeneğe bağlı fanın titreşimini ölçer. Veriyi kartın üzerinde işler (FFT ve özellik çıkarma), normal çalışmayı kendisi öğrenir ve sapma olduğunda alarm üretir. Sonuçları DBC ile tanımlı CAN mesajları olarak yayınlar. ESP32 bu CAN trafiğini WiFi üzerinden MQTT'ye taşır; PC'deki InfluxDB ve Grafana ile canlı izlenir. Son aşamada ESP32 üzerinden, CAN hattı aracılığıyla STM32'ye güvenli şekilde yeni firmware yüklenir.



Neden veriyi kartın üzerinde işliyoruz?

Ham veri $1600 \text{ Hz} \times 3 \text{ eksen} \times 2 \text{ bayt} = 9,6 \text{ KB/s}$ eder. Bu, saniyede yaklaşık 1.200 CAN çerçevesi, yani tek bir node için 500 kbit/s hattın kabaca %30'u demektir; birkaç node olsa hat tıkanır. Özetleri gönderdiğimizde ise saniyede yaklaşık 6 çerçeve yeterli olur (hattın binde birkaçı). Bu hesap, mülakatta anlatacağın en güçlü tasarım kararıdır.

Proje bittiğinde elinde olacaklar

- FreeRTOS üzerinde çalışan, FFT tabanlı titreşim analizi ve açıklanabilir anomali tespiti yapan bir sensör node'u
- DBC ile tanımlanmış, önceliklendirilmiş bir CAN mesaj sözlüğü ve iki farklı mikrodenetleyici arasında çalışan gerçek bir CAN hattı
- CAN ↔ MQTT gateway'i, Grafana dashboard'u ve MQTT'den node'a komut gönderebilen çift yönlü bir yapı
- Watchdog, bus-off kurtarma, bağlantı kopması gibi hata senaryolarının test edilmiş olduğu dayanıklı bir sistem
- CAN üzerinden, CRC32 doğrulamalı, yarıda kesilse bile kartı kullanılmaz hale getirmeyen bir firmware güncelleme mekanizması
- CI'lı bir GitHub repo'su, ölçülmüş sonuçlar içeren bir README ve 2-3 dakikalık demo videosu

Bilinçli olarak kapsam dışı

Proje 1 aya sığsın diye şunlar yapılmıyor ve README'de "sonraki adımlar" olarak yazılacak: imzalı (kriptografik) firmware doğrulama, A/B bölümlleme, birden fazla sensör node'u, makine öğrenmesi tabanlı anomali tespiti.

2. Başlamadan Önce Kontrol Listesi

Donanım siparişi verildi. Kalan kontroller ilk kurulum gününde yapılır.

Kontrol	Durum / not
☒ Discovery kart revizyonu	MB997D — kart üzerinde LIS3DSH ivmeölçer var; harici sensör gerekmez.
☐ ESP32 modeli ve USB tipi: klasik ESP32 (WROOM-32), ESP32-S3 veya ESP32-C3 uygundur.	ESP32-C2'de CAN (TWAI) birimi yoktur.
☐ ST-LINK sanal COM portu: kartı USB'ye tak, Aygıt Yöneticisi'nde "STMicroelectronics Virtual COM Port" görünüyor mu?	Görünür ve USART2 çıktısı gelirse USB-TTL gerekmez. Gelmezse CP2102 USB-TTL al (Bölüm 3.4).
☐ PC'deki boş USB port sayısı: aynı anda 3-4 cihaz takılı olacak.	ST-LINK, ESP32, lojik analizör (+ gerekirse USB-TTL). Port yetmezse harici beslemeli USB hub.
☐ Windows kullanıyorsan WSL2: Docker Desktop için gerekir.	PC tarafındaki InfluxDB/Grafana Docker'da çalışacak.
☐ 2,4 GHz WiFi ağı: PC ve ESP32 aynı ağda olmalı.	ESP32 5 GHz ve kurumsal (captive portal) ağlara bağlanamaz. Telefon hotspot'u da olur.
☐ Elindeki eski CAN modülü (MCP2515 / TJA1050)	Bu projede kullanılmıyor (5 V); yedek olarak dursun.

3. Malzeme Listesi

Liste sadeleştirildi: sadece gerçekten gerekenler satın alındı, geri kalanı evde mevcut ya da 3D baskıyla üretiliyor.

3.1 Satın alınanlar

#	Ürün	Adet	Satıcı	Tutar (TL)	Projedeki görevi
1	SN65HVD230 CAN modülü	3	Alkatronik	572,48	STM32 ve ESP32'yi CAN hattına bağlar. 2 adet kullanılır, 1 adet yedek. Pin başlıkları lehimlenecek.
2	USB lojik analizör, 24 MHz 8 kanal	1	Alkatronik	523,37	CAN ve SPI trafiğini hat üzerinde doğrulamak (PulseView).
3	Arctic P8 PWM PST, 80 mm	1	Teknobiyyotik	681,59	Titreşim kaynağı. 4 pin: PWM ile hız kontrolü, tach ile gerçek devir ölçümü. PST soketi kullanılmayacak.
4	12 V 2 A adaptör	1	shopcbnelektronik	149,00	Fanın güç kaynağı (fan ~0,1 A çeker).
5	5,5x2,1 mm DC dişi jak - klemens	1	RoboDünya	48,38	Adaptörü kesmeden fana bağlamak.
Toplam				1.974,82	Tüm kargolar ücretsiz.

3.2 Evde mevcut olanlar

Malzeme	Kullanım yeri
STM32F407 Discovery (MB997D)	Sensör node'u
ESP32 geliştirme kartı + USB kablosu	Gateway
Jumper kablolar (dişi-dişi, erkek-dişi, erkek-erkek)	Tüm bağlantılar. CAN hattı için iki jumper'ı kendi etrafında burman yeterli.
Mini breadboard	Ortak GND noktası ve tach pull-up direnci
10 kΩ direnç	Fan tach pull-up (PB4 → 3V)
Mini-USB veri kablosu	Discovery'nin ST-LINK portu
Havya + lehim, multimetre	Modül pin başlıklarını lehimlemek; 60 Ω sonlandırma ve 12 V kontrolü
M3 vida/somun, yapıştırıcı, bant	Düzenek montajı; somunlar dengesizlik ağırlığı olarak kullanılır
3D yazıcı erişimi + filament	Bölüm 6'daki parçalar

3.3 3D baskıyla üretilecekler (satın alma yok)

Taban plakası, fan dikmesi, kart yuvası, dengesizlik klipsi, **fan koruma ızgarası** ve ayaklar (TPU ile, ya da elindeki kauçuk ayaklar). Ayrıntılar Bölüm 6'da.

3.4 Sadece gerekirse sonradan alınacaklar

Ürün	Ne zaman gerekir	Tahmini TL
CP2102 USB-TTL dönüştürücü (3,3 V)	Kartın sanal COM portundan USART2 çıktısı gelmezse (Bölüm 2). İlk gün belli olur.	100-200
Harici beslemeli USB hub (4 port)	PC'de aynı anda 3-4 cihaz için yeterli port yoksa	300-600

Malzemeler gelince

- SN65HVD230 modüllerinde 120 Ω sonlandırma direnci olmalı. Gelince güç kapalıyken CANH-CANL arasını ölç: iki modül bağlıyken $\approx 60 \Omega$ görmelisin (Bölüm 5.3).
- Transceiver'ları **3,3 V**'tan besle; 5 V'a bağlama.
- Fan kablo renkleri üreticiye göre değişebilir; Bölüm 5.4'teki pin sırasına göre bağla.

4. Kurulacak Yazılımlar (hepsi ücretsiz)

Yazılım	Ne için	Not
STM32CubeMX	Pin, clock ve periferik yapılandırması; proje üretimi	Toolchain olarak CMake seç (Claude Code terminalden derleyebilsin).
STM32CubeCLT	Komut satırı derleyici, STM32CubeProgrammer CLI, GDB sunucusu	Claude Code'un derleme ve flash işlemlerini terminalden yapabilmesi için.
VS Code + STM32Cube eklentisi	Kod düzenleme ve adım adım debug	Alternatif: STM32CubeIDE. Birini seç, ikisini karıştırma.
ESP-IDF v5.x (VS Code ESP-IDF eklentisi)	ESP32 gateway firmware'i	Arduino değil ESP-IDF: TWAI, MQTT, HTTP sunucusu ve bölüm tablosu hazır.
Docker Desktop	Mosquitto, Telegraf, InfluxDB, Grafana	Windows'ta WSL2 gerekir.
PulseView (sigrok)	Lojik analizör ile CAN/SPI çözümleme	Windows'ta analizör sürücüsü Zadig ile kurulur.
Python 3.11+	Araç betikleri, DBC'den kod üretimi	Paketler: cantools, pyserial, matplotlib, paho-mqtt
Git + GitHub hesabı	Sürüm kontrolü, CI (GitHub Actions)	Repo'yu ilk günden aç.
Claude Code	Uygulama geliştirme	Bölüm 9'daki kurallarla.
MQTT Explorer	MQTT konularını canlı izleme	Gateway'i debug ederken çok işe yarar.
Seri terminal (Tera Term / PuTTY)	STM32 ve ESP32 logları	VS Code'un seri monitörü de olur.
ST-LINK sürücüsü	Programlama, debug ve sanal COM port	STM32CubeCLT/CubeProgrammer ile birlikte gelir. USB-TTL alırsan onun sürücüsü de gerekir.
Fusion 360	3D baskı parçaları	Zaten kullanıyorsun.

5. Bağlantı Şeması

5.1 STM32F407 Discovery

Sinyal	STM32 pini	Bağlandığı yer	Not
CAN1_RX	PD0	Transceiver #1 → CRX (R)	Alternatif fonksiyon AF9
CAN1_TX	PD1	Transceiver #1 → CTX (D)	Alternatif fonksiyon AF9
Besleme	3V / GND	Transceiver #1 → 3V3 / GND	5 V'a bağlama
USART2_TX	PA2	ST-LINK sanal COM portu veya USB-TTL → RX	115200 8N1
USART2_RX	PA3	ST-LINK sanal COM portu veya USB-TTL → TX	USB-TTL kullanılırsa VCC pinini bağlama; sadece GND ortak
FAN_PWM	PE9 (TIM1_CH1)	Fan pin 4 (PWM)	25 kHz, open-drain çıkış
FAN_TACH	PB4 (TIM3_CH1)	Fan pin 3 (tach) + 10 kΩ → 3V	Input capture; tur başına 2 darbe
GND	GND	Fan pin 1 ve 12 V adaptör (–)	Ortak toprak şart
İvmeölçer (kart üstü)	SPI1: PA5/PA6/PA7, CS: PE3, INT1: PE0	Dahili LIS3DSH	Kablo gerekmez
LED'ler (kart üstü)	PD12-PD15	Dahili	Yeşil: çalışıyor · Turuncu: öğreniyor · Kırmızı: alarm · Mavi: CAN trafiği
Kullanıcı butonu	PA0	Dahili	Açılıştaki basılıysa bootloader kurtarma modu (Hafta 4)

İpucu

Pinler STM32F4DISCOVERY kullanıcı kılavuzuyla (UM1472) çakışmayacak şekilde seçildi; PB6/PB9 ses çipine ait olduğu için CAN1'in PB8/PB9 seçeneği kullanılmıyor. CubeMX'te **SYS → Debug: Serial Wire** seç, yoksa PB4 JTAG'a ayrılır. CubeMX çakışma uyarısı verirse pini değiştirmeden önce kılavuza bak ve bu tabloyu güncelle.

5.2 ESP32

Sinyal	ESP32 pini	Bağlandığı yer	Not
TWAI_TX	GPIO21	Transceiver #2 → CTX (D)	
TWAI_RX	GPIO22	Transceiver #2 → CRX (R)	
Besleme	3V3 / GND	Transceiver #2 → 3V3 / GND	

Kullanma: GPIO 0, 2, 5, 12, 15 (açılış/strapping pinleri), 6–11 (dahili flash), 34–39 (sadece giriş). ESP32-S3 veya C3 kullanıyorsan pin numaralarını kartının pinout'una göre değiştir.

5.3 CAN hattı

Transceiver #1 (STM32)	Transceiver #2 (ESP32)	Kablo
CANH	CANH	Birbirine burulmuş iki jumper'ın biri
CANL	CANL	Burulmuş çiftin diğeri
GND	GND	Ayrı bir tel (ortak referans)

Sonlandırma: Hattın iki ucunda birer 120 Ω direnç olmalı; iki modülün üzerindeki dirençler bunu sağlar. **Kontrol:** güç kapalıyken CANH ile CANL arasını ölç, yaklaşık **60 Ω** görmelisin. 120 Ω görürsen bir modülde direnç yoktur, 120 Ω ekle. 40 Ω civarı görürsen hatta fazladan direnç vardır (ör. yedek modül de takılıdır), çıkar.

5.4 Fan (standart 4 pin)

Fan pini	İşlev	Bağlandığı yer
1	GND (genelde siyah)	12 V adaptör (–) ve STM32 GND
2	+12 V (genelde sarı)	12 V adaptör (+). Sadece fana.
3	Tach (genelde yeşil)	PB4 + 10 kΩ pull-up direnci 3V'a
4	PWM (genelde mavi)	PE9

Kablo renkleri üreticiye göre değişebilir; konnektördeki kilit çıkıntısına göre pin sırasına güven, emin değilsen fanın datasheet'ine bak.

İlk kez güç vermeden önce

- 12 V hiçbir kart pinine değmiyor; 12 V sadece fanın pin 2'sinde.
- Transceiver'lar 3V3'ten besleniyor (5 V'tan değil).
- Tüm GND'ler ortak: STM32, ESP32, iki transceiver, 12 V adaptörün (–) ucu.
- Güç kapalıyken CANH-CANL arası $\approx 60 \Omega$.
- USB-TTL'nin VCC pini boşta.

6. 3D Baskı Parçaları

Parça	Açıklama	Önemli ölçü / not
Taban plakası	Fan dikmesini ve kart yuvasını tek plakada birleştirir; altına kauçuk ayaklar yapıştır.	Yaklaşık 160 × 110 × 5 mm. Rijit olmalı: dolgu en az %40. Fan dikmesiyle tek parça basılırsa titreşim karta daha iyi iletilir.
Fan dikmesi	80 mm fanı dik tutar.	Fan vida delik aralığı 71,5 mm, delik çapı ~4,3 mm (fan vidaları kendinden kılavuzlu).
Kart yuvası / kısıkaçı	Discovery kartını tabana sıkıca bağlar.	Kart ölçüsünü kumpasla ölç (~97 × 66 mm). M3 vidalı kısıkaç. Kart hiç oynamamalı; gevşek bağlantı ölçümü bozar.
Dengesizlik klipsi	Fan kanadına geçen, içine M3 somun yerleşen küçük klips.	1, 2, 3 somunla farklı arıza şiddetleri. Yapıştırıcı + bantla sabitle, sonra ızgarayı tak.
Fan koruma ızgarası	Fanın önüne ve arkasına vidalanan kafes.	Güvenlik için zorunlu: kanattaki ağırlık fırlarsa önünü keser. Fan vidalarıyla birlikte sıkılır.
Ayaklar	Tabanın altına 4 adet.	TPU ile bas ya da elindeki kauçuk ayakları kullan; masa titreşimi sönmülemesin.
Modül tutucular (opsiyonel)	Transceiver ve ESP32 için küçük yuvalar.	Düzenek temiz görünür; demo videosu için değerli.

7. Teknik Tasarım (SPEC.md'nin kaynağı)

Bu bölüm repo'daki SPEC.md dosyasına birebir aktarılacak. Claude Code bu değerlerin dışına çıkmayacak; değişiklik gerekirse önce burayı (SPEC.md'yi) güncelleyeceksin.

7.1 Genel CAN parametreleri

Parametre	Değer
Bit hızı ve format	500 kbit/s, 11 bit standart ID, klasik CAN (CAN FD yok)
Bayt sırası	Little-endian (Intel), DBC'de tanımlı
STM32 bit timing	SYSCLK 168 MHz (HSE 8 MHz), APB1 42 MHz: Prescaler 6, BS1 = 11 tq, BS2 = 2 tq, SJW = 1 → 500 kbit/s, örnekleme noktası %85,7. Automatic Bus-Off Management: Enable . Automatic Retransmission: Enable .
ESP32 bit timing	500 kbit/s. ESP-IDF v5.5 ve sonrasındaki yeni TWAI sürücüsünde bit hızı doğrudan sayı olarak verilir; eski sürücüde TWAI_TIMING_CONFIG_500KBITS() makrosu kullanılır. Kullandığın ESP-IDF sürümünün dokümantasyonuna göre ilerle.
Heartbeat	Her node 500 ms'de bir; 1,5 s (3 kayıp) gelmezse karşı taraf “offline” kabul edilir
Kayıp takibi	0x100 / 0x101'deki Seq sayacı; gateway atlanan değerleri sayar ve raporlar

7.2 CAN mesaj sözlüğü (DBC)

Düşük ID yüksek öncelik demektir; bu yüzden alarm en düşük ID'yi alır. “B0-1” bayt 0 ile 1 demektir. u8/u16: işaretli, i16: işaretli tam sayı.

ID	Mesaj	Yön	DLC	Periyot	Sinyaller
0x080	VIB_ALARM	Node → GW	4	Durum değişince; aktifken 1 s'de bir	B0 Level u8 (0 NORMAL, 1 UYARI, 2 ALARM) · B1 Cause u8 bit maskesi (b0 RMS, b1 1X genlik, b2 crest) · B2-3 ZScore i16 ×0,01
0x100	VIB_FEATURES	Node → GW	8	Her pencere (~320 ms)	B0-1 Rms u16 mg · B2-3 Peak u16 mg · B4-5 Crest u16 ×0,001 · B6 Seq u8 · B7 Flags u8 (b0 baseline hazır, b1 sensör OK)
0x101	VIB_SPECTRUM	Node → GW	8	Her pencere	B0-1 DomFreq u16 ×0,01 Hz · B2-3 DomAmp u16 mg · B4-5 FanRpm u16 · B6 Seq u8 · B7 ayrılmış
0x200	NODE_CMD	GW → Node	4	İstek anında	B0 CmdId u8 (1 uyarı eşiği, 2 alarm eşiği, 3 fan duty, 4 baseline'ı yeniden öğren, 5 bootloader'a geç) · B1-2 Param u16 (eşik ×0,01 / duty %) · B3 Token u8
0x201	NODE_CMD_ACK	Node → GW	4	Komuttan sonra	B0 CmdId u8 · B1 Token u8 · B2 Result u8 (0 OK, 1 geçersiz parametre, 2 meşgul) · B3 ayrılmış
0x701	HB_NODE	Node → GW	8	500 ms	B0 State u8 (0 INIT, 1 LEARNING, 2 RUNNING, 3 FAULT) · B1 FwMajor · B2 FwMinor · B3 TEC · B4 REC · B5 ResetCause u8 (0 POR, 1 IWDG, 2 yazılım, 3 pin) · B6-7 Uptime u16 s
0x702	HB_GATEWAY	GW → Node	4	500 ms	B0 Flags u8 (b0 WiFi, b1 MQTT) · B1 FwMajor · B2 FwMinor · B3 ayrılmış
0x7E0	BL_CMD	GW → BL	8	Güncelleme sırasında	B0 Op u8 (1 START, 3 END, 4 ABORT, 5 QUERY) · START: B1-3 Size u24, B4-5 Version u16 · END: B1-4 Crc32 u32
0x7E1	BL_DATA	GW → BL	8	Güncelleme sırasında	B0-1 Seq u16 · B2-7 imajın 6 baytı (offset = Seq × 6; en fazla 384 KB imaj)
0x7E8	BL_RESP	BL → GW	8	Her cevap; bek lemedeyken 500 ms	B0 Status u8 (0 READY, 1 BUSY, 2 ACK, 3 OK, 0x10+ hata) · B1-2 Seq u16 · B3 hata detayı · B4-5 bootloader sürümü

7.3 STM32 firmware mimarisi (FreeRTOS)

Task	Öncelik	Tetikleyici	Görevi
AccelTask	Yüksek	İvmeölçer veri hazır kesmesi (INT1)	Örneği SPI'dan okur, ping-pong tampona yazar; pencere dolunca DspTask'a bildirim gönderir.
DspTask	Normal	AccelTask bildirimi	DC çıkarma, Hann penceresi, FFT, özellikler, anomali; CAN çerçevelerini CanTx kuyruğuna koyar.
CanTxTask	Normal + 1	Kuyruk	HAL_CAN_AddTxMessage'ı çağıran tek yer; mailbox doluysa bekler.
CanRxTask	Normal + 1	RX FIFO0 kesmesi → kuyruk	Komutları çözer ve uygular, ACK gönderir; gateway heartbeat'ini izler.
FanTask	Düşük	100 ms	PWM duty'yi uygular; tach periyodundan RPM hesaplar (filtreli).
HealthTask	Düşük	500 ms	Heartbeat ve LED'ler. Tüm task'lar “yaşıyorum” bildiriyse IWDG'yi besler.

- Kesme içinde sadece ...FromISR API'leri kullanılır. FreeRTOS çağıran kesmelerin NVIC önceliği sayısal olarak 5 veya daha büyük olmalı (configMAX_SYSCALL_INTERRUPT_PRIORITY).
- FreeRTOS SysTick'i kullandığı için HAL timebase **TIM6**'ya taşınır.
- bxCAN'da en az bir filtre bankası yapılandırılmadan hiçbir mesaj alınmaz; başlangıçta “hepsini kabul et” filtresi kurulur.
- UART logları tek bir sahip üzerinden (kuyrukla) yazılır; task'lar doğrudan printf çağırmaz.
- IWDG zaman aşımı yaklaşık 2 s. Reset sebebi açılışta RCC bayraklarından okunup heartbeat'e konur.

7.4 Sinyal işleme

Parametre	Değer (LIS3DSH, kart üstü)
Örnekleme hızı (ODR)	1600 Hz
FFT boyu	1024 nokta (CMSIS-DSP arm_rfft_fast_f32)
Pencere süresi / çözünürlük	0,64 s / 1,5625 Hz
Nyquist frekansı	800 Hz
Güncelleme aralığı	~320 ms (%50 örtüşme)
Ölçek	±2 g

- **RMS:** pencerenin DC'si çıkarılmış sinyalinin karekök ortalaması (mg).
- **Tepe (peak):** penceredeki mutlak en büyük değer (mg). **Crest factor:** tepe / RMS; darbeli arızalarda yükselir.
- **Baskın frekans:** DC hariç en büyük FFT tepesi; komşu bin'lerle parabolik interpolasyon yapılır.
- **1X genlik:** tach'tan bulunan dönme frekansındaki FFT genliği. Dönme frekansı = RPM / 60 = tach frekansı / 2.
- **Eksen seçimi:** öğrenme başında en yüksek RMS'li eksen seçilip sabitlenir.
- Arctic P8 PWM PST 200–3000 RPM aralığında döner; 3000 RPM'de 1X = 50 Hz. Fanda kanat geçiş frekansı (kanat sayısı × 1X) da görünebilir; kanatları sayıp beklenen frekansı hesapla. Bu, README'de anlatılacak iyi bir detaydır.
- DSP süresi DWT cycle counter ile ölçülür ve README'ye yazılır.

7.5 Anomali tespiti

1. **Öğrenme:** ilk 30 saniyede (~90 pencere) RMS ve 1X genliğinin ortalaması (μ) ve standart sapması (σ) çıkarılır. σ 'ya bir alt sınır uygulanır (ör. 2 mg), çok düzgün sinyalde sıfıra yakın değere bölünmesin.
2. **Çalışma:** her pencerede $z = (x - \mu) / \sigma$ hesaplanır; iki özellikten büyük olan kullanılır.
3. **UYARI:** art arda 3 pencere $z \geq 3$. **ALARM:** art arda 3 pencere $z \geq 6$.
4. **Normale dönüş:** art arda 5 pencere $z < 2$ (histerezis; alarm durmadan açılıp kapanmasın).
5. Fan hızı değiştiğinde (komut 3) veya komut 4 geldiğinde baseline yeniden öğrenilir. Eşikler komut 1 ve 2 ile değiştirilebilir.

7.6 ESP32 gateway

ESP-IDF v5.x. Task'lar: CAN alma, MQTT'ye yayın, MQTT komutlarını CAN'a iletme, heartbeat ve izleme, firmware güncelleme (HTTP sunucusu + bootloader durum makinesi). CAN çözme/kodlama DBC'den cantools ile üretilen ortak C koduyla yapılır; STM32 ile aynı dosyalar kullanılır.

MQTT konusu	Yön	İçerik (JSON)
vibnode/1/features	GW → broker	rms_mg, peak_mg, crest, seq, ts
vibnode/1/spectrum	GW → broker	dom_freq_hz, dom_amp_mg, fan_rpm, seq, ts
vibnode/1/alarm	GW → broker	level, cause, z, ts
vibnode/1/status	GW → broker (retained)	online, state, fw, tec, rec, reset_cause, uptime_s, lost_frames
gateway/status	GW → broker (retained, LWT)	online, wifi_rssi, fw, can_state
vibnode/1/cmd	broker → GW	Ör. {"cmd": "set_fan_duty", "value": 60}
vibnode/1/cmd_ack	GW → broker	cmd, token, result
vibnode/1/fw	GW → broker	state, progress_pct, error

- partitions.csv içinde yüklenen STM32 firmware'i için 1 MB'lık bir "stm_fw" veri bölümü tanımlanır.
- TWAI hata durumları izlenir; bus-off durumunda sürücünün kurtarma fonksiyonuyla otomatik toparlanır (fonksiyon adı ESP-IDF sürümüne göre değişir).
- MQTT bağlantısı yokken son ~500 mesaj RAM'deki halka tamponda tutulur, bağlantı gelince gönderilir.
- WiFi bilgileri ve broker adresi menuconfig/NVS'de tutulur, koda gömülmez. Zaman damgası SNTP ile; internet yoksa Telegraf alış zamanını kullanır.

7.7 PC tarafı (Docker)

Servis	İmaj	Görev
Mosquitto	eclipse-mosquitto:2	MQTT broker. mosquitto.conf: listener 1883 ve allow_anonymous true (sadece lab ağında).
Telegraf	telegraf	mqtt_consumer + json_v2 ayrıştırıcı → InfluxDB
InfluxDB	influxdb:2	Zaman serisi veritabanı (bucket: vibnode)
Grafana	grafana/grafana	Veri kaynağı ve dashboard JSON'u repo'dan provisioning ile otomatik yüklenir.

Dashboard panelleri: RMS trendi ve eşik çizgileri; baskın frekans ile tach'tan hesaplanan dönme frekansı aynı grafikte; alarm durumu zaman çizelgesi; node ve gateway online durumu; TEC/REC; firmware sürümü; kayıp çerçeve sayısı. Windows güvenlik duvarında 1883 portuna izin ver; PC'nin IP adresi değişmesin diye modemde DHCP rezervasyonu yap.

7.8 Bootloader ve uzaktan güncelleme

Bölge	Adres aralığı	Flash sektörü	Boyut
Bootloader	0x0800 0000 – 0x0800 7FFF	0-1	32 KB
Metadata	0x0800 8000 – 0x0800 BFFF	2	16 KB (magic, boyut, CRC32, sürüm)
Uygulama	0x0800 C000 – 0x080F FFFF	3-11	976 KB

Açılıştaki karar sırası

1. RTC backup register'da “güncelleme isteği” işareti varsa → güncelleme modu (işaret temizlenir).
2. Kullanıcı butonu (PA0) basılıysa → güncelleme modu (kurtarma yolu).
3. Metadata geçerli, uygulamanın CRC32'si tutuyor ve ilk kelime (stack pointer) SRAM aralığında (0x2000 0000 – 0x2002 0000) ise → uygulamaya atla.
4. Hiçbiri değilse → güncelleme modu.

Güncelleme akışı

1. Gateway NODE_CMD 5 gönderir → uygulama backup register'a işaret yazar ve yazılımsal reset atar.
2. Bootloader güncelleme moduna girer, 500 ms'de bir BL_RESP READY yayınlar.
3. START (boyut, sürüm) → metadata ve gereken uygulama sektörleri silinir (önce BUSY, sonra READY). 128 KB'lık sektör silme 1-2 s sürer; gateway'de START zaman aşımı 10 s olmalı.
4. BL_DATA (seq, 6 bayt) → flash'a yazılır → ACK (seq). Dur-bekle yöntemi: gateway her ACK'i bekler; 100 ms zaman aşımı, 3 tekrar.
5. END (crc32) → flash'taki imajın CRC'si hesaplanır. Tutarsa metadata yazılır, OK döner, reset → yeni uygulama açılır. Tutmazsa hata kodu döner, bootloader'da kalınır.

Neden kart asla kullanılamaz hale gelmez: metadata START'ta silinir ve sadece doğrulama başarılı olduktan sonra yazılır. Akışın herhangi bir yerinde güç kesilirse kart açılıştaki bootloader'da kalır ve yeniden güncellenebilir.

Hata kodları: 0x10 CRC uyuşmazlığı · 0x11 sıra hatası · 0x12 boyut hatası · 0x13 flash silme/yazma hatası · 0x14 zaman aşımı.

Atlama sırası

Kullanılan periferyleri kapat (CAN, GPIO, RCC) → SysTick'i durdur → NVIC'te tüm kesmeleri kapat ve bekleyenleri temizle → SCB→VTOR = 0x0800C000 → MSP'yi uygulamanın stack değerine ayarla → kesmeleri (PRIMASK) geri aç → uygulamanın reset vektörüne atla. PRIMASK'ı açmayı unutursan uygulamada hiçbir kesme çalışmaz.

Uygulama tarafında gerekenler

- Linker script'te FLASH ORIGIN = 0x0800C000, LENGTH = 976K.
- system_stm32f4xx.c'de USER_VECT_TAB_ADDRESS tanımlanır, VECT_TAB_OFFSET = 0xC000.
- Derleme sonrası .bin üretilir; tools/fw_pack.py boyutu ve CRC32'yi hesaplar.
- **Hafta 1-3 boyunca uygulama normal şekilde 0x08000000'de çalışır.** Taşıma sadece Hafta 4'te yapılır (linker + VTOR değişikliği).
- Geliştirme kolaylığı: debug derlemesinde BL_DEV_ALLOW_UNVERIFIED bayrağı, metadata geçersiz olsa da stack pointer geçerliyse uygulamaya atlar. Release derlemesinde kapalıdır.
- CRC32 yazılımla (zlib uyumlu) hesaplanır. F4'ün donanım CRC birimi farklı bir varyant kullandığı için ESP32 ve Python ile aynı sonucu vermez.

8. Repo Yapısı

vibnode/	
├─ CLAUDE.md	Claude Code kuralları (Bölüm 9)
├─ SPEC.md	Teknik sözleşme (Bölüm 7)
├─ README.md	
├─ dbc/vibnode.dbc	CAN mesajlarının tek doğruluk kaynağı
├─ common/	Saf C, donanımdan bağımsız, PC'de test edilir
│ ├─ can/	cantools ile DBC'den üretilen kod
│ ├─ dsp_features.c/.h	
│ ├─ anomaly.c/.h	
│ ├─ crc32.c/.h	
│ └─ bl_protocol.h	
├─ tests/	Host unit testleri (Unity + CMake/ctest)
├─ firmware/	
│ ├─ app/	CubeMX (CMake) projesi; uygulama kodu app/ altında
│ └─ bootloader/	Ayrı CubeMX projesi
├─ gateway/	ESP-IDF projesi
├─ infra/	docker-compose.yml, mosquitto, telegraf, grafana
├─ tools/	fw_pack.py, seri veri çizici, CAN log çözücü
├─ mechanical/	Fusion 360 ve STL dosyaları
├─ docs/	
│ ├─ decisions/	Karar notları
│ └─ images/	

CI (GitHub Actions)

Üç iş: (1) host unit testleri ve cppcheck, (2) STM32 uygulama ve bootloader derlemesi (arm-none-eabi-gcc + CMake), (3) ESP-IDF derlemesi (resmi espressif/idf Docker imajıyla). README'ye “build passing” rozeti eklenir.

Yazılacak karar notları (docs/decisions/)

Her biri 2-3 paragraf: bağlam, seçenekler, seçim ve gerekçesi. Mülakat hazırlığın bunlar olacak.

1. Veriyi uçta işleme (Bölüm 1'deki bant genişliği hesabı)
2. CAN ID önceliklendirmesi (alarm neden en düşük ID)
3. z-score + histerezis (neden ML yerine açıklanabilir bir yöntem)
4. Flash haritası ve metadata'nın ayrı sektörde tutulması
5. Dur-bekle bootloader protokolü (basitlik ve güvenilirlik ile hız arasındaki takas)
6. Yazılım CRC32 seçimi
7. ESP-IDF seçimi (Arduino yerine)

9. Claude Code ile Çalışma Yöntemi

9.1 CLAUDE.md şablonu

Repo'nun köküne koy. Komutları kendi kurulumuna göre düzelt.

```
# vibnode – Claude Code kuralları

## Proje
STM32F407 titreşim node'u + ESP32 CAN-MQTT gateway + CAN bootloader.
Teknik sözleşme: SPEC.md. CAN mesajları: dbc/vibnode.dbc (tek doğruluk kaynağı).

## Komutlar
- Host testleri : cmake -S tests -B build/tests && ctest --test-dir build/tests
- STM32 derleme : cd firmware/app && cmake --preset Debug && cmake --build --preset Debug
- STM32 flash   : STM32_Programmer_CLI -c port=SWD -w <elf> -v -rst
- ESP32         : idf.py -C gateway build flash monitor
- DBC'den kod   : cd common/can && python -m cantools generate_c_source ../../dbc/vibnode.dbc

## Kurallar
1. DBC, pinler, clock, CAN bit timing ve flash haritası benim onayım olmadan değişmez.
2. CubeMX dosyalarında sadece USER CODE BEGIN/END blokları içine yaz.
   Uygulama kodu firmware/app/app/ altındaki kendi dosyalarında durur.
3. common/ saf C'dir: HAL veya FreeRTOS başlığı içeremez; her fonksiyonun host testi olur.
4. Kesme içinde sadece ...FromISR API'leri. CAN sadece CanTxTask üzerinden gönderilir.
5. Her değişiklikten sonra ilgili derlemeyi ve testleri çalıştır; kırmızıysa iş bitmemiştir.
6. Donanım davranışını tahmin etme; emin değilsen sor ya da benden ölçüm iste.
7. Görev sonunda özetle: ne değişti, nasıl test edildi, donanımda neyi doğrulamam gerekiyor.
```

9.2 Her görev için çalışma döngüsü

1. Görevi tek cümleyle ve ölçülebilir bir kabul kriteriyle yaz (örnek aşağıda).
2. Claude Code'a önce plan yaptır; planı oku ve onayla, sonra uygulat.
3. Derleme ve testler yeşil mi bak; donanım gerektiriyorsa kendin doğrula.
4. Diff'i oku. Anlamadığın her satırı sor; anlamadığın kodu commit'leme.
5. Açıklayıcı bir mesajla commit'le. Önemli bir kararsa docs/decisions/ altına not düş.

Örnek görev tanımı

Görev: common/crc32.c ve crc32.h dosyalarını yaz.

- Standart CRC-32 (IEEE 802.3, zlib uyumlu): yansıtılmış polinom 0xEDB88320, başlangıç 0xFFFFFFFF, son XOR 0xFFFFFFFF, tablo tabanlı.
- Parça parça hesaplama için crc32_init / crc32_update / crc32_final API'si.

Kabul kriteri:

- tests/test_crc32.c: "123456789" için 0xCBFA3926.
- Rastgele 1 KB veri için Python zlib.crc32 ile aynı sonuç.
- ctest yeşil. Bu dosyalar ve testleri dışında hiçbir şeye dokunma.

9.3 İş bölümü

Sen yaparsın	Claude Code yapar
CubeMX (.ioc) yapılandırması; pin, clock ve bit timing değerlerinin datasheet'ten doğrulanması	Sürücü sarmalayıcıları, task iskeletleri, CAN katmanı
Lehim, kablolama, 60 Q ve voltaj ölçümleri	DSP ve anomali kodu, host unit testleri
Lojik analizörle hat üzerinde doğrulama ve debug	ESP-IDF gateway, MQTT, HTTP yükleme sayfası
3D tasarım, baskı ve montaj	docker-compose, Telegraf, Grafana provisioning
Fiziksel testler: ağırlık, kablo çekme, güç kesme	Python araçları (fw_pack, seri çizici), CI
Kod incelemesi, karar notları, demo videosu	README ve dokümantasyon taslakları

İpucu

Agent'ların en sık hata yaptığı yerler: clock ağacı, CAN bit timing, pin alternatif fonksiyonları, kesme öncelikleri ve CubeMX'in yeniden ürettiği dosyalar. Bu beş konuda Claude Code'un yazdığını her zaman datasheet'e veya CubeMX'e karşı kontrol et.

10. Haftalık Plan ve Öğrenme Rehberi

Her haftanın sonunda “bitti” kriterlerinin hepsi işaretlenmeden bir sonraki haftaya geçme.

Öğrenme rehberi nasıl kullanılır

Görev tablolarındaki **Konu** sütunu, o görevden önce çalışman gereken konu kartını gösterir (ör. K1.3). Konu kartları her haftanın görev listesinden hemen sonra gelir. Her kartta kavramın ne olduğu ve bu projede neden önemli olduğu, hangi görevde kullanıldığı, kaynaklar ve kendini test edeceğin sorular var.

1. Görevden önce ilgili kartın açıklamasını oku ve ★ **işaretli** kaynağı tamamla. Diğer kaynaklar derinleşmek içindir.
2. Görevi Claude Code'a verirken kavramı kendi cümlelerinle açıkla. Açıklayamıyorsan henüz hazır değilsin; kaynağa geri dön.
3. Görev bitince kartın “Kendini test et” sorularını yazılı olarak cevapla ve Ek A'daki cevap anahtarıyla karşılaştı.
4. Cevaplarını repo'da **docs/learning-notes.md** dosyasında tut. Bu dosya mülakat hazırlığın olacak.

★ işaretli kaynaklar haftada yaklaşık 4–6 saat tutar. Kaynak başlıkları tıklanabilir bağlantıdır; tam adresler Ek B'de listelenmiştir.

Hafta 1 — Altyapı ve CAN hattı

Hedef: İki mikrodenetleyici gerçek bir CAN hattı üzerinden konuşuyor ve bağlantı kopmasını algılıyor.

#	Görev	Konu	Kim
1	Kalan kontrolleri yap (Bölüm 2): ESP32 modeli, sanal COM port, USB portları, WSL2, WiFi	—	Sen
2	Yazılımları kur (Bölüm 4), GitHub repo'sunu aç	K1.1	Sen
3	CLAUDE.md ve SPEC.md'yi bu dokümandan oluştur	K1.1	Birlikte
4	DBC dosyasını yaz, cantools ile C kodu üret, pack/unpack host testleri	K1.5	Claude Code (sen incele)
5	common/crc32 ve testleri; CI iskeleti (host testleri + cppcheck)	K1.1, K4.5	Claude Code
6	3D parçaları tasarla ve bas (Bölüm 6)	—	Sen
7	<i>Malzeme gelince:</i> pin başlıklarını lehimle, kablolamayı yap, 60 Ω ölç	K1.2	Sen
8	CubeMX: 168 MHz clock, Serial Wire debug, USART2, CAN1 (PD0/PD1, Bölüm 7.1), FreeRTOS, HAL timebase TIM6, CMake çıktısı	K1.3, K1.7, K1.8	Sen
9	STM32: UART log, CAN katmanı (filtre, başlatma, kesmeler), CanTx/CanRx task'ları, heartbeat, ivmeölçer WHO_AM_I okuması	K1.2, K1.4, K1.6, K1.7	Claude Code
10	ESP32: ESP-IDF projesi, TWAI, heartbeat gönderme ve izleme, seri log	K1.9	Claude Code
11	PulseView'da CAN çözümlemesiyle trafiği doğrula	K1.10	Sen

Bitti sayılması için

- ☐ İki node birbirinin heartbeat'ini görüyor ve logluyor.
- ☐ CAN kablosu çekilince iki taraf da 1,5 s içinde “offline” diyor; geri takınca kendiliğinden düzeliyor.
- ☐ PulseView'da 0x701 ve 0x702 ID'leri ve verileri okunuyor.
- ☐ İvmeölçerin WHO_AM_I kaydı 0x3F (LIS3DSH) olarak okunuyor.
- ☐ CI yeşil.
- ☐ K1.x kartlarının test sorularını cevapladın.

Not

Görev 1–6 donanım gerektirmez; kargolar yoldayken bunları bitir. Bu haftanın konuları diğer haftalardan fazla, çünkü projenin temeli burada atılıyor.

Hafta 1 konuları

K1.1 · Git ve GitHub iş akışı

Nedir, neden önemli? Git, kodunun her halini (commit) kayıt altında tutar; bir şey bozulursa geri dönebilirsin. Agent'larla çalışırken bu çok daha önemli: her görevi ayrı bir commit olarak alırsan, Claude Code'un yaptığı bir değişikliği tek komutla geri alabilirsin. GitHub Actions ise her push'ta kodu otomatik derleyip test eden bir "sürekli entegrasyon" (CI) hizmetidir.

Projede: Görev 2, 5

Kaynaklar

- ★ [Pro Git \(Türkçe çeviri\) — Bölüm 1-3](#) — Kitap, 3 sa
- [GitHub Actions dokümantasyonu](#) — Doküman, 1 sa

Kendini test et

1. git revert ile git reset arasındaki fark nedir? Paylaşılan bir dalda hangisi güvenlidir?
2. Her görevi neden ayrı commit'lersin?

K1.2 · CAN veri yolunun temelleri

Nedir, neden önemli? CAN, birçok cihazın iki kablo (CANH ve CANL) üzerinden, merkezi bir yönetici olmadan konuştuğu bir yoldur. Bilgi iki tel arasındaki gerilim farkıyla taşındığı için gürültüye dayanıklıdır. Her mesajın bir ID'si vardır; iki node aynı anda konuşmaya başlarsa düşük ID'li olan kazanır, kaybeden ise veri kaybetmeden sonra tekrar dener (arbitration). Alıcı node'lar her çerçevenin sonunda bir ACK biti basar; hatta ACK verecek kimse yoksa gönderici mesajı hatalı sayar. Hattın iki ucundaki 120 Ω dirençler yansımaları önler.

Projede: Görev 7-11; Bölüm 5.3 ve 7.2

Kaynaklar

- ★ [CSS Electronics — CAN Bus Explained: A Simple Intro](#) — Okuma + video, 30 dk
- ★ [Texas Instruments SLOA101 — Introduction to the Controller Area Network \(CAN\)](#) — Uygulama notu, 1 sa (Bölüm 3-5)

Kendini test et

1. İki node aynı anda 0x080 ve 0x100 ID'li mesaj göndermeye başlarsa ne olur?
2. Hatta yalnızca tek bir node varken gönderdiği mesaja ne olur?
3. Güç kapalıyken CANH-CANL arası 120 Ω ölçüyorsa ne anlamalısın?

K1.3 · CAN bit timing (bit zamanlaması)

Nedir, neden önemli? Her CAN biti küçük zaman dilimlerine (time quantum, tq) bölünür: 1 senkronizasyon dilimi + BS1 + BS2. Bit hızı = CAN saat frekansı / (Prescaler × (1 + BS1 + BS2)). Hattaki değer BS1'in sonunda okunur; bu noktaya örnekleme noktası denir ve genelde bitin %75-87,5'ine konur. Hattaki bütün node'lar aynı bit hızında olmalıdır; yanlış bir hesap, hattın hiç çalışmamasının en sık sebebidir. Bizim ayarımız: 42 MHz / (6 × (1 + 11 + 2)) = 500 kbit/s, örnekleme noktası (1 + 11) / 14 = %85,7.

Projede: Görev 8; Bölüm 7.1

Kaynaklar

- ★ [ST Community — CAN \(bxCAN\) bit time configuration on STM32 MCUs](#) — Okuma, 30 dk
- [CAN bit timing hesaplayıcı \(bittiming.can-wiki.info\)](#) — Araç, gerektiğinde

Kendini test et

1. APB1 = 42 MHz iken 250 kbit/s için geçerli bir ayar bul.
2. Örnekleme noktası neden bitin sonuna yakın seçilir?

K1.4 · CAN hata yönetimi

Nedir, neden önemli? Her CAN node'u iki sayaç tutar: gönderme hataları (TEC) ve alma hataları (REC). Sayaçlar arttıkça node önce "error passive", TEC 255'i aşınca da "bus-off" durumuna düşer ve hattı tamamen çekilir. Bu, arızalı bir node'un tüm hattı kilitlemesini engelleyen bir güvenlik mekanizmasıdır. STM32'de Automatic Bus-Off Management açıksa node, hatta belirli sayıda boş bit gördükten sonra kendiliğinden geri döner.

Projede: Görev 9-11; Hafta 3 görev 5

Kaynaklar

★ [CSS Electronics — CAN Bus Errors Explained](#) — Okuma, 40 dk

Kendini test et

1. Bus-off'a düşen bir STM32 node'u nasıl hatta geri döner?
2. Heartbeat mesajlarındaki TEC/REC değerleri sana ne söyler?

K1.5 · DBC dosyası ve koddan üretim

Nedir, neden önemli? DBC, bir CAN ağındaki tüm mesajları ve bu mesajların içindeki sinyalleri tanımlayan standart bir metin dosyasıdır. Her sinyal için başlangıç biti, uzunluk, bayt sırası, ölçek (factor), ofset ve birim yazılır. cantools bu dosyadan C kodu (pack/unpack fonksiyonları) üretir. Böylece STM32 ve ESP32 aynı dosyadan üretilen kodu kullanır ve iki tarafın mesajı farklı yorumlaması imkânsız hale gelir. İşteki ECU tecrübenle doğrudan bağlantılı bir konu.

Projede: Görev 4; Bölüm 7.2

Kaynaklar

★ [CSS Electronics — CAN DBC File Explained](#) — Okuma, 30 dk

○ [cantools dokümantasyonu](#) — Doküman, gerektilçe

Kendini test et

1. Rms = 1234 mg, u16, ölçek 1, little-endian. B0 ve B1 baytları ne olur?
2. ZScore = -2,50 değeri x0,01 ölçekli i16 sinyale nasıl kodlanır?

K1.6 · FreeRTOS temelleri: task, öncelik, kuyruk

Nedir, neden önemli? RTOS, işlemcinin zamanını birden fazla görev (task) arasında paylaştırır. Her task'ın bir önceliği vardır; hazır olan en yüksek öncelikli task çalışır ve daha düşük olanı kesebilir (preemption). Task'lar birbirine veriyi kuyruk (queue) ile güvenli şekilde aktarır. Kesme (ISR) içinde bekleyen fonksiyonlar çağrılmaz; bu yüzden ISR'lerde sadece ...FromISR ile biten API'ler kullanılır.

Projede: Görev 9; Bölüm 7.3

Kaynaklar

★ [Digi-Key — Introduction to RTOS \(12 bölümlük video serisi\)](#) — Video, ≈1,5 sa (Bölüm 1-3, 5, 9)

○ [Mastering the FreeRTOS Real Time Kernel \(ücretsiz kitap\)](#) — Kitap, ilgili bölümler

Kendini test et

1. Neden CAN'a mesaj yazan tek bir task (CanTxTask) var?
2. ISR içinde xQueueSend (FromISR olmayan sürüm) çağırırsan ne olur?

K1.7 · Cortex-M kesmeleri ve FreeRTOS kesme öncelikleri

Nedir, neden önemli? Cortex-M'de kesme önceliğinde küçük sayı yüksek öncelik demektir; bu çoğu kişiye ters gelir. FreeRTOS, configMAX_SYSCALL_INTERRUPT_PRIORITY ile kesmeleri iki gruba ayırır: bu sınırın üstünde (sayısal olarak küçük) önceliğe sahip kesmeler FreeRTOS API'si çağırılmaz ama hiç maskelenmez. CAN, SPI gibi FreeRTOS'a veri gönderen kesmelerin önceliği sayısal olarak bu sınıra eşit ya da büyük olmalıdır (CubeMX varsayılanında 5).

Projede: Görev 9; Bölüm 7.3

Kaynaklar

★ [FreeRTOS — Running the RTOS on an ARM Cortex-M Core \(kesme öncelikleri\)](#) — Doküman, 30 dk

○ [Memfault Interrupt — A Practical Guide to ARM Cortex-M Exception Handling](#) — Okuma, 45 dk

Kendini test et

1. CAN RX kesmesinin önceliğini 2 yaparsan ve içinde xQueueSendFromISR çağırırsan ne olur?

K1.8 · STM32 saat ağacı, CubeMX ve kart pinleri

Nedir, neden önemli? Discovery kartındaki 8 MHz kristal (HSE), PLL ile 168 MHz sistem saatine çıkarılır. Bu saat bölünerek çevre birimlerine dağıtılır: CAN1 ve USART2 APB1 hattında (42 MHz), TIM1 APB2 zamanlayıcı saatinde (168 MHz). Bit hızı ve PWM frekansı hesaplarının hepsi bu değerlere dayanır. Kartın hangi pininin başka bir bileşene (ses çipi, LED, ivmeölçer) bağlı olduğu UM1472'deki pin tablosunda yazar.

Projede: Görev 8; Bölüm 5.1

Kaynaklar

- ★ [ST UM1472 — STM32F4DISCOVERY kullanıcı kılavuzu](#) — Kılavuz, pin tablosu bölümü
- [ST RM0090 — STM32F407 referans kılavuzu](#) — Kılavuz, RCC bölümü, başvuru

Kendini test et

1. CAN1 çevre birimi hangi saat hattından beslenir ve bizim ayarımızda frekansı nedir?
2. PB9'u neden CAN için kullanmıyoruz?

K1.9 · ESP32 TWAI (CAN) sürücüsü

Nedir, neden önemli? ESP32'nin içinde CAN protokolünü işleyen bir denetleyici (TWAI) vardır; sadece hatta bağlanmak için dışarıdan transceiver gerekir. Sürücünün "normal" modu hattaki diğer node'lardan ACK bekler; test için ACK beklemeden self-test modu da vardır. ESP-IDF v5.5 ve sonrasında sürücü API'si yenilendi: bit hızı doğrudan sayı olarak verilir. İnternetteki eski örnekler eski API'yi kullanabilir; kullandığın ESP-IDF sürümünün dokümantasyonuna bak.

Projede: Görev 10

Kaynaklar

- ★ [ESP-IDF — TWAI \(CAN\) sürücü dokümantasyonu](#) — Doküman, 1 sa

Kendini test et

1. Transceiver'lar gelmeden ESP32 tarafında TWAI kodunu nasıl test edebilirsin?

K1.10 · Lojik analizörle CAN çözümleme

Nedir, neden önemli? Lojik analizör, dijital hatları zaman içinde kaydedip bitleri görmeni sağlar; PulseView'daki CAN çözümleyici de bu bitleri ID, veri ve ACK olarak okur. Önemli nokta: problemler CANH/CANL diferansiyel hattına değil, transceiver'ın mikrodenetleyici tarafındaki TX veya RX pinine (3,3 V lojik) bağlanır. Örneklem hızı bit hızının en az 8 katı olmalı; 500 kbit/s için 4–12 MHz uygundur. Kod ile hat arasındaki gerçeği gösteren bu araç, agent'ların sana veremeyeceği bir bilgidir.

Projede: Görev 11

Kaynaklar

- ★ [Atadiat — Saleae klonu 24 MHz 8 kanal lojik analizör uygulamalı test](#) — Okuma, 30 dk
- [sigrok / PulseView resmi sitesi](#) — Araç, gerektikçe

Kendini test et

1. Problemleri neden CANH/CANL yerine RX pinine bağlarsın?
2. PulseView'da mesajlar görünüyor ama her çerçevenin sonunda ACK yok. Ne düşünürsün?

Hafta 2 — Sensör, sinyal işleme ve fan

Hedef: Fana ağırlık takıldığında kart bunu kendi başına tespit edip CAN'a alarm basıyor.

#	Görev	Konu	Kim
1	İvmeölçer sürücüsü: yapılandırma (ODR, ölçek), veri hazır kesmesi, AccelTask	K2.1	Claude Code
2	Ham veriyi UART'tan PC'ye akıt, tools/ altındaki Python betiğiyle çiz; kartı elle sallayıp doğrula	K2.1, K2.2	Birlikte
3	common/dsp_features: pencere, FFT (CMSIS-DSP), özellikler. Sentetik sinyal testleri (ör. 50 Hz, 100 mg sinüs → baskın frekans $50 \pm 1,6$ Hz, $RMS \approx 70,7$ mg)	K2.3, K2.4, K2.5, K2.9	Claude Code
4	common/anomaly: öğrenme, z-score, histerezis ve testleri	K2.7, K2.9	Claude Code
5	Fan: TIM1 PWM (25 kHz), TIM3 input capture ile RPM, FanTask	K2.8	Claude Code
6	Düzenegi monte et: taban, fan, izgara, kart yuvası, ayaklar	K2.6	Sen
7	DspTask'ı bağla; 0x080, 0x100, 0x101 mesajlarını yayınla; DSP süresini DWT ile ölç	K2.5, K1.6	Claude Code
8	Ağırlık denemeleri (0 / 1 / 2 / 3 somun) ve üç farklı fan hızında ölçüm	K2.6	Sen

Bitti sayılması için

- ☐ Üç farklı fan hızında ölçülen 1X frekansı, tach'tan hesaplanan dönme frekansı ile $\pm 1,6$ Hz uyumlu.
- ☐ Ağırlık takılınca yaklaşık 5 s içinde ALARM, çıkarılınca normale dönüyor.
- ☐ DSP süresi ve CPU yükü ölçüldü ve not edildi.
- ☐ Host testleri CI'da geçiyor.
- ☐ K2.x kartlarının test sorularını cevapladın.

Hafta 2 konuları

K2.1 · SPI ve LIS3DSH ivmeölçer

Nedir, neden önemli? SPI, saat (SCK), veri (MOSI/MISO) ve seçim (CS) hatlarından oluşan hızlı bir seri arayüzdür. Sensörle konuşmak için önce CS'yi düşürür, ardından okumak istediğin kaydın adresini gönderirsin. LIS3DSH'te ilk baytın en üst biti 1 ise okuma yapılır. WHO_AM_I kaydı (0x0F) sensörün kimliğini (0x3F) verir; bağlantıyı doğrulamanın ilk adımıdır. Örnekleme hızı (ODR) ve ölçek kontrol kayıtlarından ayarlanır; "veri hazır" kesmesi yeni örnek geldiğinde INT1 pinini tetikler.

Projede: Görev 1-2

Kaynaklar

- ★ [SparkFun — Serial Peripheral Interface \(SPI\)](#) — Okuma, 30 dk
- ★ [ST — LIS3DSH ürün sayfası ve datasheet](#) — Datasheet, SPI ve kayıt haritası bölümleri

Kendini test et

- WHO_AM_I kaydını okumak için SPI üzerinden hangi baytı gönderirsin?
- Neden sensörü sürekli sorgulamak (polling) yerine veri hazır kesmesi kullanıyoruz?

K2.2 · Örnekleme, Nyquist ve aliasing

Nedir, neden önemli? Bir sinyali saniyede fs kez örneklersen, ancak fs/2'ye (Nyquist frekansı) kadar olan bileşenleri doğru görebilirsin. Daha yüksek frekanslar yok olmaz; daha düşük bir frekansmış gibi görünür (aliasing) ve seni yanıltır. LIS3DSH'i 1600 Hz'de çalıştırdığımız için 800 Hz'e kadar güvenle bakabiliriz; fanımızın 1X frekansı en fazla 50 Hz olduğundan bol pay var.

Projede: Görev 3; Bölüm 7.4

Kaynaklar

- ★ [Steven W. Smith — The Scientist and Engineer's Guide to DSP \(ücretsiz kitap\)](#) — Kitap, Bölüm 3

Kendini test et

- fs = 1600 Hz iken 1000 Hz'lik bir titreşim spektrumunda nerede görünür?

K2.3 · Fourier dönüşümü ve FFT

Nedir, neden önemli? Fourier dönüşümü, zaman içinde karmaşık görünen bir sinyali onu oluşturan frekanslara ayırır. FFT bunu hızlı hesaplayan algoritmadır. N örnekli bir FFT, 0'dan $f_s/2$ 'ye kadar $N/2$ kutucuk (bin) üretir; her kutucuk f_s/N genişliğindedir. Bizim ayarımızda bu $1600 / 1024 = 1,5625$ Hz eder. Bir makinedeki arıza genelde belirli bir frekansta tepe olarak görünür; FFT bunu görünür kılar.

Projede: Görev 3, 7

Kaynaklar

- ★ [3Blue1Brown — But what is the Fourier Transform? A visual introduction](#) — Video, 20 dk
- [Steven W. Smith — The Scientist and Engineer's Guide to DSP \(ücretsiz kitap\)](#) — Kitap, Bölüm 8 ve 12

Kendini test et

1. $f_s = 1600$ Hz, $N = 1024$ iken 50 Hz hangi kutucuğa düşer?
2. Pencereyi 2048 örneğe çıkarırsan ne kazanır, ne kaybedersin?

K2.4 · Pencereleme ve spektral sızıntı

Nedir, neden önemli? FFT, pencerenin sonsuza kadar tekrarlandığını varsayar. Pencere tam sayıda periyot içermiyorsa uçlarda bir kopukluk oluşur ve bir frekansın enerjisi komşu kutucuklara yayılır (spektral sızıntı). Sinyali uçlarda sıfıra inen bir pencere fonksiyonuyla (Hann) çarpmak bu sorunu büyük ölçüde azaltır. Bedeli, tepelerin biraz genişlemesi ve genliğin düşmesidir; Hann penceresinde genliği doğru okumak için sonucu 2 ile çarpmak gerekir.

Projede: Görev 3

Kaynaklar

- ★ [National Instruments — Understanding FFTs and Windowing](#) — Uygulama notu, 45 dk
- [Steven W. Smith — The Scientist and Engineer's Guide to DSP \(ücretsiz kitap\)](#) — Kitap, Bölüm 9

Kendini test et

1. Hann penceresi uygulanmış 100 mg genlikli bir sinüsün FFT tepesi düzeltme yapılmazsa yaklaşık kaç mg çıkar?

K2.5 · CMSIS-DSP ile FFT

Nedir, neden önemli? CMSIS-DSP, ARM'ın Cortex-M için optimize ettiği sinyal işleme kütüphanesidir. `arm_rfft_fast_f32` gerçek sayılı bir FFT hesaplar. Çıktı özel bir biçimdedir: ilk iki eleman DC ve Nyquist bileşenlerinin gerçek kısımlarıdır, sonrası gerçek/sanal çiftler halinde gelir. Genlik için `arm_cmplx_mag_f32` kullanılır. Kütüphane taşınabilir C olduğu için PC'de de derlenebilir; böylece aynı DSP kodunu kart olmadan test edebilirsiniz.

Projede: Görev 3, 7

Kaynaklar

- ★ [CMSIS-DSP — Real FFT F32 fonksiyonları](#) — Doküman, 30 dk

Kendini test et

1. `arm_rfft_fast_f32` çıktısında `pOut[1]` neyi içerir?

K2.6 · Titreşim analizinin temelleri

Nedir, neden önemli? Dönen bir parçada dengesizlik (unbalance), dönme frekansında (1X) belirgin ve radyal bir tepe üretir; genliği hızın karesiyle artar. RMS sinyalin genel enerjisini, tepe değer en büyük sıçramayı, crest factor (tepe/RMS) ise darbeli olayları gösterir. Fanda ayrıca kanat sayısı $\times$ 1X frekansında kanat geçiş tepesi görülebilir. Bu kavramlar, işte deniz platformu motorları için hedeflediğin kestirimci bakımın temelidir.

Projede: Görev 7–8

Kaynaklar

- ★ [Valmet — An introduction to troubleshooting with vibration signatures](#) — Okuma, 20 dk
- [Fabrico — Vibration spectrum analysis \(arıza-frekans tablosu\)](#) — Okuma, 30 dk

Kendini test et

1. Kanada ağırlık eklediğinde spektrumda hangi tepenin büyümesini beklersin?
2. Fan hızını ikiye katlarsan dengesizlik kuvveti nasıl değişir?

K2.7 · İstatistiksel anomali tespiti: z-score ve histerezis

Nedir, neden önemli? z-score, bir ölçümün normal ortalamadan kaç standart sapma uzakta olduğunu söyler: $z = (x - \mu) / \sigma$. Önce normal çalışmayı öğrenip (μ ve σ), sonra her yeni değer z-score'una bakarız. Tek bir sıçramada alarm vermemek için art arda birkaç pencere şartı (debounce), alarmın açılıp kapanmaması için de farklı açma/kapama eşikleri (histerezis) kullanılır. Basit ama açıklanabilir bir yöntem: mülakatta her kararını savunabilirsin.

Projede: Görev 4

Kaynaklar

★ [NIST/SEMATECH e-Handbook — Detection of Outliers](#) — Okuma, 30 dk

Kendini test et

1. σ neredeyse sıfırsa ne olur ve nasıl önlenir?
2. Histerezis olmasaydı $z \approx 3$ civarında dolaşan bir sinyalde ne görürdün?

K2.8 · Timer: PWM üretimi ve input capture

Nedir, neden önemli? STM32 timer'ları bir sayaçtır: saat frekansı prescaler (PSC) ile bölünür, sayaç 0'dan ARR'ye kadar sayar ve başa döner. PWM'de sayaç CCR değerini geçince çıkış değişir; frekans = saat / ((PSC + 1) × (ARR + 1)). Input capture ise bir kenar geldiğinde sayacın o anki değerini kaydeder; iki kenar arasındaki farktan periyot hesaplanır. 4 pinli fanlar 25 kHz PWM bekler ve tach hattından tur başına 2 darbe verir; tach çıkışı open-collector olduğu için pull-up direnci gerekir.

Projede: Görev 5

Kaynaklar

★ [ST AN4776 — How to use general-purpose timer peripheral on STM32 MCUs](#) — Uygulama notu, PWM ve input capture bölümleri

★ [Noctua — PWM specifications white paper \(4 pinli fan standardı\)](#) — Teknik doküman, 20 dk

○ [SparkFun — Pulse Width Modulation](#) — Okuma, 15 dk

Kendini test et

1. TIM1 saati 168 MHz, PSC = 0 iken 25 kHz PWM için ARR kaç olmalı?
2. Tach sinyali 100 Hz ölçülüyorsa fan kaç RPM döner?

K2.9 · Unit test ve donanımsız doğrulama

Nedir, neden önemli? Donanımdan bağımsız kodu (FFT, özellik çıkarma, anomali, CRC) PC'de derleyip test edersen hataları saniyeler içinde bulursun. Unity, C için hafif bir test çatısıdır. Sentetik sinyal üretmek (örneğin bilinen genlik ve frekansta bir sinüs) test için en güçlü araçtır: sonucun ne olması gerektiğini önceden bildiğin için karşılaştırma yapabilirsin. Bu testler CI'da her push'ta çalışır ve agent'ın yaptığı değişiklikleri güvenle kabul etmeni sağlar.

Projede: Görev 3-4

Kaynaklar

★ [ThrowTheSwitch — Unity \(C için unit test çatısı\)](#) — Doküman, 30 dk

Kendini test et

1. 100 mg genlikli saf bir sinüsün RMS değeri ne olmalı?
2. Neden agent'ın yazdığı DSP kodunu önce karta değil PC'deki teste sokarsın?

Hafta 3 — Gateway, dashboard ve dayanıklılık

Hedef: Veri Grafana'da canlı; sistem hatalara karşı kendi kendine toparlanıyor.

#	Görev	Konu	Kim
1	ESP32: WiFi + MQTT, CAN → JSON yayını (DBC'den üretilen kodla)	K3.1, K3.2	Claude Code
2	MQTT komutu → CAN (0x200), ACK → MQTT; LWT ve retained durum mesajları	K3.1	Claude Code
3	infra/: docker-compose (Mosquitto, Telegraf, InfluxDB, Grafana) ve provisioning	K3.3, K3.4	Claude Code
4	Grafana dashboard'unu tasarla, JSON'unu repo'ya kaydet	K3.3	Birlikte
5	Dayanıklılık: STM32'de IWDG + task check-in, reset sebebi; ESP32'de bus-off kurtarma, MQTT halka tamponu, kayıp çerçeve sayacı	K3.5, K1.4, K3.2	Claude Code
6	Hata senaryolarını test et (aşağıdaki kriterler)	K3.6	Sen
7	README taslağı ve ilk demo videosu	—	Birlikte

Bitti sayılması için

- ☐ Grafana'da canlı veri akıyor, alarm paneli çalışıyor.
- ☐ MQTT'den fan hızı komutu gönderince 1 s içinde RPM değişiyor ve ACK geliyor.
- ☐ CAN kablosu çekilip takılınca sistem müdahalesiz toparlanıyor.
- ☐ WiFi 60 s kesilince veri kaybolmuyor (tampondan gönderiliyor) ya da kayıp sayısı raporlanıyor.
- ☐ Bir task kasıtlı kilitlenince IWDG kartı resetliyor, heartbeat'te ResetCause = IWDG görünüyor.
- ☐ K3.x kartlarının test sorularını cevapladın.

Not

Bu noktada proje tamamdır. Hafta 4 bonus niteliğindedir; kayarsa bu sürümü yayınla ve başvurulara başla.

Hafta 3 konuları

K3.1 · MQTT: yayımla/abone ol modeli

Nedir, neden önemli? MQTT'de cihazlar birbirine doğrudan bağlanmaz; herkes bir aracı sunucuya (broker) bağlanır. Veri "konu" (topic) adlı yollara yayınlanır, ilgilenenler o konulara abone olur. QoS mesajın teslim garantisini belirler. Retained mesajlar, bir konunun son değerini yeni abonelere hemen verir. Last Will (LWT), bağlantısı beklenmedik şekilde kopan bir cihaz adına broker'ın yayınladığı son mesajdır; gateway'in çevrimdışı olduğunu bu sayede anlarız.

Projede: Görev 1-2; Bölüm 7.6

Kaynaklar

- ★ [HiveMQ — MQTT Essentials serisi \(tüm bölümlerin listesi\)](#) — Okuma, 1-2 sa (Bölüm 1-9)
- [HiveMQ — MQTT Essentials Part 8: Retained Messages](#) — Okuma, 15 dk

Kendini test et

- gateway/status konusunu neden retained yayınlıyoruz?
- LWT mesajı ne zaman yayınlanır?

K3.2 · ESP-IDF'de WiFi ve MQTT istemcisi

Nedir, neden önemli? ESP-IDF'de WiFi ve MQTT olay tabanlı çalışır: bağlantı kuruldu, koptu veya mesaj geldi gibi durumlar bir olay döngüsü üzerinden bildirilir. Bağlantı koptuğunda yeniden bağlanmayı ve o arada gelen verileri tamponlamayı sen tasarlarsın. WiFi şifresi ve broker adresi gibi bilgiler koda gömülmez, menuconfig veya NVS'de tutulur.

Projede: Görev 1-2, 5

Kaynaklar

- ★ [ESP-IDF — ESP-MQTT dokümantasyonu](#) — Doküman, 45 dk

Kendini test et

- MQTT bağlantısı kopukken gelen CAN verisine ne olmalı?

K3.3 · Zaman serisi veri hattı: Telegraf → InfluxDB → Grafana

Nedir, neden önemli? Telegraf, MQTT konularını dinleyip JSON'u ayrıştırır ve InfluxDB'ye yazar. InfluxDB zaman damgalı verileri saklar: measurement (tablo gibi), tag (filtrelemede kullanılan etiket, örneğin node kimliği) ve field (ölçülen değer, örneğin rms_mg). Grafana bu veriyi sorgulayıp panellerde gösterir. Provisioning ile veri kaynağı ve dashboard'lar dosyadan yüklenir; böylece dashboard'un da repo'da sürüm kontrolünde olur.

Projede: Görev 3-4

Kaynaklar

- ★ [InfluxData — Telegraf ile MQTT'den veri toplama örneği](#) — Doküman, 30 dk
- [Grafana — Provisioning dokümantasyonu](#) — Doküman, 30 dk

Kendini test et

1. node kimliği tag mi field mı olmalı? rms_mg?

K3.4 · Docker Compose

Nedir, neden önemli? Docker Compose, birden fazla servisi (Mosquitto, Telegraf, InfluxDB, Grafana) tek bir YAML dosyasıyla tanımlayıp tek komutla ayağa kaldırır. Her servisin imajı, portları, kalıcı verisi (volume) ve yapılandırma dosyaları bu dosyada yazar. Projeyi başka bir bilgisayarda çalıştırmak "docker compose up" kadar kolaylaşır; bu da README'nin gücünü artırır.

Projede: Görev 3

Kaynaklar

- ★ [Docker Compose dokümantasyonu](#) — Doküman, 45 dk
- [Mosquitto — mosquitto.conf başvuru sayfası](#) — Doküman, listener ve allow_anonymous kısmı

Kendini test et

1. InfluxDB verisi konteyner silince neden kaybolmaz?

K3.5 · Watchdog tasarımı

Nedir, neden önemli? Bağımsız watchdog (IWDG), kendi saatiyle (LSI) sayan ve düzenli beslenmezse işlemciyi resetleyen bir donanımdır. Yanlış kullanım, onu ana döngüden körü körüne beslemektir; o durumda bir task kilitlese bile watchdog fark etmez. Doğru yöntem: her kritik task belli aralıklarla "yaşıyorum" bildirir; watchdog'u besleyen tek task, sadece herkes bildirdiyse besler. Açılıştan RCC kayıtlarından reset sebebi okunup raporlanır.

Projede: Görev 5-6

Kaynaklar

- ★ [ST RM0090 — STM32F407 referans kılavuzu](#) — Kılavuz, IWDG ve RCC_CSR bölümleri

Kendini test et

1. Watchdog'u neden sadece tek bir yerden beslersin?

K3.6 · Hata enjeksiyonu ve dayanıklılık testi

Nedir, neden önemli? Bir sistemin güvenilir olduğunu göstermenin yolu, onu bilerek bozmaktır: CAN kablosunu çekmek, WiFi'yi kapatmak, bir task'ı kilitlemek. Her senaryo için beklenen davranışı önceden yaz, testi yap, sonucu kaydet. Savunma sanayinde bu yaklaşım doğrulamanın temelidir ve README'de "test edilen arıza senaryoları" tablosu olarak çok güçlü durur.

Projede: Görev 6

Kaynaklar

- [CSS Electronics — CAN Bus Errors Explained](#) — Okuma, tekrar: deneyler bölümü

Kendini test et

1. CAN kablosu 10 saniye çekili kalırsa STM32 ve ESP32 tarafında ne görmeyi beklersin?

Hafta 4 — Bootloader, uzaktan güncelleme ve teslim

Hedef: Web sayfasından yüklenen firmware CAN üzerinden karta yazılıyor; bozuk dosya ve güç kesintisi kartı bozmuyor.

#	Görev	Konu	Kim
1	Uygulamayı 0x0800C000'e taşı (linker + VTOR), normal çalıştığını doğrula	K4.1, K4.2	Sen + Claude Code
2	Bootloader projesi: açılış kararı, atlama, flash silme/yazma, CRC32, metadata	K4.2, K4.3, K4.4, K4.5	Claude Code
3	Bootloader CAN protokolü (0x7E0 / 0x7E1 / 0x7E8) ve host'ta protokol testleri	K4.6	Claude Code
4	tools/fw_pack.py (boyut + CRC32)	K4.5	Claude Code
5	ESP32: stm_fw bölümü, HTTP yükleme sayfası, güncelleme durum makinesi, ilerlemenin MQTT'ye yayını	K4.7	Claude Code
6	Güncelleme testleri (aşağıda) ve OTA hızının ölçümü	K4.6	Sen
7	README'yi tamamla, demo videosunu çek, CV satırını ekle	—	Birlikte

Bitti sayılması için

- ☐ Web sayfasından v1.1 yüklenince heartbeat'te 1.1 görünüyor.
- ☐ Bir baytı bozulmuş dosya reddediliyor; kart bootloader'da bekliyor ve doğru dosyayla güncellenebiliyor.
- ☐ Güncellenmenin ortasında güç kesilince kart bootloader'da açılıyor ve kurtarılabilir.
- ☐ Butonla kurtarma modu çalışıyor.
- ☐ OTA hızı (KB/s) ölçüldü ve README'ye yazıldı.
- ☐ K4.x kartlarının test sorularını cevapladın.

Hafta 4 konuları

K4.1 · Bellek haritası ve linker script

Nedir, neden önemli? Linker script, derlenen kodun hangi adrese yerleşeceğini tanımlar: FLASH ve RAM bölgelerinin başlangıcı (ORIGIN) ve boyutu (LENGTH). STM32F407'nin flash'ı eşit olmayan sektörlerden oluşur: 4 × 16 KB, 1 × 64 KB ve 7 × 128 KB. Bootloader ile uygulamayı ayırmak için uygulamanın ORIGIN değerini 0x0800C000'e taşıyoruz; sektör sınırlarına göre yerleşim yapmak şarttır çünkü flash sektör sektör silinir.

Projede: Görev 1; Bölüm 7.8

Kaynaklar

- ★ [Memfault Interrupt — From Zero to main\(\): Demystifying Firmware Linker Scripts](#) — Okuma, 45 dk
- [ST RM0090 — STM32F407 referans kılavuzu](#) — Kılavuz, Flash bölümü, sektör tablosu

Kendini test et

1. Metadata'yı neden uygulamayla aynı sektöre koymuyoruz?

K4.2 · Vector table, VTOR ve açılış sırası

Nedir, neden önemli? Cortex-M açılışta vector table'ın ilk kelimesini yığın işaretçisi (SP), ikinci kelimesini reset handler adresi olarak okur. Kesme geldiğinde işlemci hangi fonksiyona gideceğini bu tablodan bulur; tablonun yeri VTOR kaydında durur. Uygulama 0x0800C000'e taşındığında VTOR da oraya ayarlanmazsa kesmeler bootloader'ın tablosuna gider ve sistem çöker.

Projede: Görev 1-2

Kaynaklar

- ★ [Memfault Interrupt — A Practical Guide to ARM Cortex-M Exception Handling](#) — Okuma, vector table bölümü
- [Memfault Interrupt — How to Write a Bootloader from Scratch](#) — Okuma, jump bölümü

Kendini test et

1. Uygulama imajının ilk 8 baytı ne içerir?
2. Bootloader uygulamanın geçerli olduğunu anlamak için ilk kelimeye neden bakar?

K4.3 · Bootloader tasarımı

Nedir, neden önemli? Bootloader, her açılışta önce çalışan küçük bir programdır: uygulamanın geçerli olup olmadığına karar verir, geçerliyse ona atlar, değilse güncelleme bekler. Asıl zorluk atlamadır: bootloader'ın açtığı çevre birimleri, kesmeler ve SysTick uygulamaya "kirli" şekilde devredilmemelidir. Bootloader'ı küçük ve değişmez tutmak, tüm sistemin güvenilirliğinin temelidir.

Projede: Görev 2

Kaynaklar

★ [Memfault Interrupt — How to Write a Bootloader from Scratch](#) — Okuma, 1 sa

Kendini test et

1. Atlamadan önce PRIMASK'ı geri açmazsan ne olur?

K4.4 · Flash programlama

Nedir, neden önemli? Flash'a yazmadan önce kilidi açmak ve hedef sektörü silmek gerekir; silme tüm bitleri 1 yapar, yazma ise sadece bitleri 1'den 0'a çevirebilir. 128 KB'lık bir sektörü silmek 1–2 saniye sürebilir ve bu sırada o flash bankından kod çalıştırmak bekletilir. Bu yüzden bootloader silme sırasında BUSY bildirir ve gateway uzun bir zaman aşımı kullanır.

Projede: Görev 2–3

Kaynaklar

★ [ST RM0090 — STM32F407 referans kılavuzu](#) — Kılavuz, Embedded Flash memory interface bölümü

Kendini test et

1. Silmeden bir kelimeyi 0x00 yazıp sonra 0xFF yazmaya çalışırsan ne olur?

K4.5 · CRC

Nedir, neden önemli? CRC, bir veri bloğunun bozulup bozulmadığını anlamak için hesaplanan kısa bir kontrol değeridir. Gönderici hesaplar, alıcı aynı hesabı yapıp karşılaştırır. Aynı "CRC-32" adı altında farklı varyantlar vardır (polinom, bit yansıtma, başlangıç değeri, son XOR); iki taraf aynı varyantı kullanmazsa sonuçlar tutmaz. Standart CRC-32 için "123456789" dizisinin sonucu 0xCBF43926'dır; bunu testinde referans olarak kullanırsın.

Projede: Hafta 1 görev 5; Görev 2–4

Kaynaklar

★ [Ross Williams — A Painless Guide to CRC Error Detection Algorithms](#) — Okuma, 1 sa (Bölüm 1–7 ve 13–16)

Kendini test et

1. CRC32 ile imzalamak aynı şey mi?

K4.6 · Güvenilir güncelleme mimarisi

Nedir, neden önemli? Sahadaki bir cihazın güncellemesi yarıda kesilebilir, dosya bozuk gelebilir veya yeni sürüm hatalı olabilir. İyi bir tasarım bu durumların hiçbirinde cihazı kullanılamaz hale getirmez. Bizim tasarımımda geçerlilik bilgisi (metadata) en son, doğrulamadan sonra yazılır; bu yüzden herhangi bir anda güç kesilse kart bootloader'da kalır. A/B bölümlendirme ve kriptografik imza bir sonraki adımdır.

Projede: Görev 2, 6

Kaynaklar

★ [Memfault Interrupt — Device Firmware Update Cookbook](#) — Okuma, 45 dk

○ [Memfault Interrupt — Secure Firmware Updates with Code Signing](#) — Okuma, 45 dk

Kendini test et

1. Güncelleme sırasında son DATA paketinden hemen sonra güç kesilirse ne olur?

K4.7 · ESP32 bölüm tablosu ve HTTP sunucusu

Nedir, neden önemli? ESP32'nin flash'ı bölüm tablosuyla (partitions.csv) parçalara ayrılır: uygulama, NVS ve bizim ekleyeceğimiz STM32 firmware'i için bir veri bölümü. Yüklenen .bin önce bu bölüme yazılır, sonra oradan parça parça CAN'a gönderilir; böylece HTTP bağlantısının hızı ile CAN'ın hızı birbirinden ayrılır. esp_http_server, dosya yükleme sayfasını sunmak için yeterlidir.

Projede: Görev 5

Kaynaklar

★ [ESP-IDF — Partition Tables](#) — Doküman, 30 dk

○ [ESP-IDF — HTTP Server](#) — Doküman, 30 dk

Kendini test et

1. Dosyayı RAM yerine neden flash bölümüne yazıyoruz?

11. Sık Karşılaşılan Sorunlar

Belirti	Sebebi	Çözüm
STM32 hiç CAN mesajı almıyor	bxCAN filtresi yapılandırılmamış	En az bir filtre bankasını “hepsini kabul et” olarak kur; HAL_CAN_Start ve bildirim aktivasyonunu çağır.
Mesajlar gitmiyor, TEC sürekli artıyor	Hatta ACK verecek ikinci node yok ya da bit hızı uyuşmuyor	İki node da açık ve 500 kbit/s olmalı; Bölüm 7.1'deki değerleri kontrol et.
FreeRTOS açınca HAL zaman aşımaları bozuluyor	SysTick çakışması	HAL timebase'i TIM6'ya taşı.
Kesmeden kuyruğa yazınca hard fault / assert	FromISR olmayan API veya yanlış NVIC önceliği	Sadece ...FromISR kullan; kesme önceliği sayısal olarak ≥ 5 .
CubeMX yeniden üretince kod silindi	Kod USER CODE blokları dışına yazılmış	Uygulama kodunu ayrı dosyalarda tut (CLAUDE.md kural 2).
CRC STM32 ile ESP32/Python'da farklı	F4 donanım CRC birimi farklı bir CRC-32 varyantı	Yazılım CRC32 (zlib uyumlu) kullan; host testinde zlib.crc32 ile karşılaştır.
Bootloader'dan atlayınca uygulama donuyor	VTOR ayarlanmamış, PRIMASK kapalı kalmış veya SysTick açık	Bölüm 7.8'deki atlama sırasını uygula.
START komutundan sonra zaman aşımı	128 KB'lık sektör silme 1-2 s sürüyor	Bootloader BUSY göndersin; gateway'de START zaman aşımı 10 s.
Kart debugger olmadan açılmıyor (Hafta 4)	Metadata geçersiz	fw_pack ile metadata yaz ya da debug derlemesinde BL_DEV_ALLOW_UNVERIFIED kullan.
ESP32 WiFi'ye bağlanmıyor	5 GHz ya da kurumsal ağ	2,4 GHz WPA2 ağı veya telefon hotspot'u kullan.
ESP32 broker'a bağlanamıyor	Mosquitto 2 varsayılan olarak sadece localhost'u dinler; güvenlik duvarı	mosquitto.conf'a listener 1883 ve allow_anonymous true ekle; 1883 portuna izin ver.
İvmeölçer yanlış veya hiç veri vermiyor	SPI ayarı veya CS pini hatalı	WHO_AM_I 0x3F dönmeli. SPI1 modu (CPOL/CPHA), hızı ve CS (PE3) ayarını kontrol et.
Titreşim sinyali çok zayıf	Kart gevşek bağlı ya da masa titreşimi sönümlüyor	Kartı rijit sabitle; tabanı kauçuk ayaklarla masadan ayır.
PulseView'da CAN çözülüyor, anlamsız bitler görünüyor	Problar CANH/CANL'ye bağlı ya da örnekleme hızı düşük	Probu transceiver'ın RX (veya TX) pinine bağla; örnekleme hızını en az 4 MHz yap (K1.10).
FFT sonucunda DC ve en üst kutucuk tuhaf	arm_rfft_fast_f32 çıktı biçimi	pOut[0] DC, pOut[1] Nyquist gerçek kısmıdır; genlik hesabında bunları ayrı ele al (K2.5).
İnternetteki ESP32 TWAI örneği derlenmiyor	ESP-IDF sürümleri arasında TWAI API'si değişti	Kullandığın sürümün dokümantasyonundaki örneği temel al (K1.9).
Fan PWM'e tepki vermiyor	Ortak GND yok ya da pin push-pull ayarlı	12 V GND ile STM32 GND'yi birleştir; pini open-drain ayarla. Bazı fanlar %20'nin altında durur.

12. Teslim

12.1 README içeriği

- Tek cümlelik özet ve kısa bir demo GIF'i
- Mimari diyagram, donanım listesi ve kablolama
- CAN mesaj sözlüğü (DBC'ye bağlantı)
- Tasarım kararları (docs/decisions/ özetleri)
- **Ölçülen sonuçlar:** DSP süresi ve CPU yükü, frekans doğruluğu, alarm tepki süresi, OTA hızı
- Test edilen hata senaryoları ve sonuçları
- **Bilinen sınırlamalar:** firmware imzası yok (bütünlük var, kimlik doğrulama yok), A/B bölümlleme yok, tek sensör node'u
- Kurulum ve çalıştırma adımları

12.2 Demo videosu (2-3 dakika)

1. Düzeneğin kısa tanıtımı ve mimari
2. Normal çalışma: Grafana'da canlı veri
3. Grafana/MQTT'den fan hızını değiştirme → frekans takibi
4. Kanada ağırlık takma → UYARI ve ALARM
5. CAN kablosunu çekme → offline tespiti ve otomatik toparlanma
6. Web sayfasından yeni firmware yükleme → heartbeat'te yeni sürüm
7. Bozuk dosya yükleme → reddedilmesi

12.3 CV satırı

CV'ye eklenecek madde

STM32F407 üzerinde FreeRTOS tabanlı bir titreşim izleme node'u geliştirdim: CMSIS-DSP ile uçta FFT ve özellik çıkarma, istatistiksel anomali tespiti yapıyor ve sonuçları DBC ile tanımlı CAN mesajlarıyla yayınlıyor. ESP32 ile CAN-MQTT gateway'i kurup verileri InfluxDB/Grafana'da görselleştirdim; CAN üzerinden CRC32 doğrulamalı, kesintiye dayanıklı bir firmware güncelleme mekanizması (bootloader) geliştirdim.

12.4 Proje sonrası (opsiyonel)

Zaman kalırsa sırasıyla: bootloader'a ECDSA imza doğrulaması (micro-ecc gibi küçük bir kütüphaneyle), A/B bölümlleme ve otomatik geri dönüş, fanın yerine fırçasız drone motoru + pervane ve DroneCAN desteği ("İHA motor/pervane sağlık izleme node'u").

Ek A. Cevap Anahtarı

Önce kendi cevabını yaz, sonra buraya bak. Cevabın farklı ama mantığı doğruysa sorun yok; amaç ezber değil anlamak.

Hafta 1

K1.1 · Git ve GitHub iş akışı

1. *git revert ile git reset arasındaki fark nedir? Paylaşılan bir dalda hangisi güvenlidir?*

revert, eski commit'i geri alan yeni bir commit oluşturur ve geçmişi korur; reset ise dalın işaretçisini geri taşır ve geçmişi değiştirir. Paylaşılan dalda revert güvenlidir.

2. *Her görevi neden ayrı commit'lersin?*

Hatalı bir değişikliği diğerlerini bozmadan geri almak, hangi değişikliğin hangi hataya yol açtığını bulmak (git bisect) ve kod incelemesini küçük parçalar halinde yapmak için.

K1.2 · CAN veri yolunun temelleri

1. *İki node aynı anda 0x080 ve 0x100 ID'li mesaj göndermeye başlarsa ne olur?*

Bitler aynı anda hatta yazılır; dominant (0) bit recessive (1) bite baskın gelir. 0x080'in ID'sinde daha erken bir 0 biti olduğu için o kazanır. 0x100'ü gönderen node kaybettiğini fark eder, yayını durdurur ve hat boşalınca tekrar dener. Veri kaybolmaz.

2. *Hatta yalnızca tek bir node varken gönderdiği mesaja ne olur?*

ACK alamaz, mesajı hatalı sayar ve tekrar tekrar gönderir. Hata sayacı (TEC) artar; node sonunda error-passive duruma düşer.

3. *Güç kapalıyken CANH-CANL arası 120 Ω ölçüyorsa ne anlamalısın?*

Hatta sonlandırma direnci iki değil, bir tane var. Diğer uca bir 120 Ω daha eklenmeli; doğru değer $\approx 60 \Omega$ 'dur.

K1.3 · CAN bit timing (bit zamanlaması)

1. *APB1 = 42 MHz iken 250 kbit/s için geçerli bir ayar bul.*

Örneğin Prescaler 12, BS1 = 11, BS2 = 2: $42 \text{ MHz} / (12 \times 14) = 250 \text{ kbit/s}$, örnekleme noktası yine %85,7.

2. *Örnekleme noktası neden bitin sonuna yakın seçilir?*

Sinyalin hat boyunca yayılıp oturması ve kenar geçişlerinin bitmesi için zaman tanır; böylece bit kararlı haldeyken okunur.

K1.4 · CAN hata yönetimi

1. *Bus-off'a düşen bir STM32 node'u nasıl hatta geri döner?*

ABOM açıksa, hatta 128 kez 11 ardışık recessive bit gördükten sonra otomatik olarak; kapalıysa yazılımın CAN birimini yeniden başlatması gerekir.

2. *Heartbeat mesajlarındaki TEC/REC değerleri sana ne söyler?*

Hattın sağlığını: sıfıra yakın değerler sağlıklı hat demektir; sürekli artan değerler kablo, sonlandırma veya bit hızı sorununa işaret eder.

K1.5 · DBC dosyası ve koddan üretim

1. *Rms = 1234 mg, u16, ölçek 1, little-endian. B0 ve B1 baytları ne olur?*

$1234 = 0x04D2 \rightarrow B0 = 0xD2, B1 = 0x04$.

2. *ZScore = -2,50 değeri $\times 0,01$ ölçekli i16 sinyale nasıl kodlanır?*

$-2,50 / 0,01 = -250 \rightarrow i16$ olarak $0xFF06$; little-endian baytlar $0x06, 0xFF$.

K1.6 · FreeRTOS temelleri: task, öncelik, kuyruk

1. *Neden CAN'a mesaj yazan tek bir task (CanTxTask) var?*

Donanım mailbox'larına birden fazla task'ın aynı anda erişmesi yarış durumuna (race condition) yol açar. Tek sahip, erişimi kilitsiz ve öngörülebilir kılar; diğer task'lar mesajlarını kuyruğa bırakır.

2. *ISR içinde xQueueSend (FromISR olmayan sürüm) çağırırsan ne olur?*

Fonksiyon beklemeye geçmeye çalışabilir ve zamanlayıcıyı bozar; FreeRTOS'un assert kontrolleri açıksa program durur, değilse öngörülemez çökmeler yaşanır.

K1.7 · Cortex-M kesmeleri ve FreeRTOS kesme öncelikleri

1. *CAN RX kesmesinin önceliğini 2 yaparsan ve içinde xQueueSendFromISR çağırırsan ne olur?*

Öncelik 2, FreeRTOS'un maskeleyebildiği sınırın (5) üstünde kalır; kritik bölümler sırasında da çalışabilir ve kernel yapılarını bozabilir. configASSERT açıksa hemen yakalanır.

K1.8 · STM32 saat ağacı, CubeMX ve kart pinleri

1. *CAN1 çevre birimi hangi saat hattından beslenir ve bizim ayarımızda frekansı nedir?*

APB1; $168 \text{ MHz} / 4 = 42 \text{ MHz}$.

2. *PB9'u neden CAN için kullanmıyoruz?*

Kart üzerindeki ses çipinin I2C hattına bağlı; çakışma olur.

K1.9 · ESP32 TWAI (CAN) sürücüsü**1. Transceiver'lar gelmeden ESP32 tarafında TWAI kodunu nasıl test edebilirsin?**

Self-test / no-ACK modunda kendi gönderdiği mesajı alarak; böylece yapılandırma, filtreler ve task yapısı donanım gelmeden doğrulanır.

K1.10 · Lojik analizörle CAN çözümleme**1. Problemleri neden CANH/CANL yerine RX pinine bağlarsın?**

CANH/CANL tek başına lojik seviye taşımaz, bilgi ikisinin farkındadır; RX pini ise hattaki durumu 0/1 olarak mikrodenetleyici gerilim seviyesinde verir.

2. PulseView'da mesajlar görünüyor ama her çerçevenin sonunda ACK yok. Ne düşünürsün?

Hatta ACK verecek ikinci bir node çalışmıyor ya da bit hızı uyuşmuyor; diğer node'un açık olduğunu ve bit timing ayarlarını kontrol et.

Hafta 2**K2.1 · SPI ve LIS3DSH ivmeölçer****1. WHO_AM_I kaydını okumak için SPI üzerinden hangi baytı gönderirsin?**

0x0F | 0x80 = 0x8F; ardından gelen baytta 0x3F okunmalı.

2. Neden sensörü sürekli sorgulamak (polling) yerine veri hazır kesmesi kullanıyoruz?

Örnekler eşit aralıklarla alınır (FFT için şart) ve işlemci boşuna meşgul edilmez.

K2.2 · Örnekleme, Nyquist ve aliasing**1. $f_s = 1600$ Hz iken 1000 Hz'lik bir titreşim spektrumunda nerede görünür?**

Nyquist'in (800 Hz) üstünde kaldığı için $1600 - 1000 = 600$ Hz'de sahte bir tepe olarak görünür.

K2.3 · Fourier dönüşümü ve FFT**1. $f_s = 1600$ Hz, $N = 1024$ iken 50 Hz hangi kutucuğa düşer?**

$50 / 1,5625 = 32 \rightarrow 32$. kutucuk.

2. Pencereyi 2048 örneğe çıkarırsan ne kazanır, ne kaybedersin?

Frekans çözünürlüğü ikiye katlanır ($0,78$ Hz) ama pencere $1,28$ s'ye uzar; ani değişimlere tepki yavaşlar ve bellek kullanımı artar.

K2.4 · Pencereleme ve spektral sızıntı**1. Hann penceresi uygulanmış 100 mg genlikli bir sinüsün FFT tepesi düzeltme yapılmazsa yaklaşık kaç mg çıkar?**

Yaklaşık 50 mg; Hann penceresinin genlik kazancı $0,5$ 'tir, bu yüzden 2 ile çarparak düzeltilir.

K2.5 · CMSIS-DSP ile FFT**1. `arm_rfft_fast_f32` çıktısında `pOut[1]` neyi içerir?**

Nyquist frekansındaki ($f_s/2$) bileşenin gerçek kısmını; normal bir karmaşık sayının sanal kısmı değildir.

K2.6 · Titreşim analizinin temelleri**1. Kanada ağırlık eklediğinde spektrumda hangi tepenin büyümesini beklersin?**

$1X$ tepesinin (dönme frekansı, 3000 RPM'de 50 Hz).

2. Fan hızını ikiye katlarsan dengesizlik kuvveti nasıl değişir?

Merkezkaç kuvveti hızın karesiyle arttığı için yaklaşık dört katına çıkar.

K2.7 · İstatistiksel anomali tespiti: z-score ve histerezis**1. σ neredeyse sıfırda ne olur ve nasıl önlenir?**

Çok küçük bir sapma bile devasa bir z-score üretir ve sahte alarm verir; σ için bir alt sınır (σ_{min}) konarak önlenir.

2. Histerezis olmasaydı $z \approx 3$ civarında dolaşan bir sinyalde ne görürdün?

Alarm sürekli açılıp kapanırdı (chattering).

K2.8 · Timer: PWM üretimi ve input capture**1. `TIM1` saati 168 MHz, `PSC = 0` iken 25 kHz PWM için `ARR` kaç olmalı?**

$168\,000\,000 / 25\,000 = 6720 \rightarrow ARR = 6719$.

2. `Tach` sinyali 100 Hz ölçülüyorsa fan kaç RPM döner?

Tur başına 2 darbe: $100 / 2 = 50$ tur/s $\rightarrow 3000$ RPM.

K2.9 · Unit test ve donanımsız doğrulama**1. 100 mg genlikli saf bir sinüsün RMS değeri ne olmalı?**

$100 / \sqrt{2} \approx 70,7$ mg.

2. Neden agent'ın yazdığı DSP kodunu önce karta değil PC'deki teste sokarsın?

Hata ayıklama çok daha hızlıdır, sonuç deterministiktir ve bilinen bir cevaba karşı doğrulanır; kartta aynı hatayı bulmak saatler alabilir.

Hafta 3

K3.1 · MQTT: yayınla/abone ol modeli

1. gateway/status konusunu neden retained yayınlıyoruz?

Dashboard sonradan bağlansa bile gateway'in son durumunu hemen görsün diye.

2. LWT mesajı ne zaman yayınlanır?

İstemci düzgün bir DISCONNECT göndermeden bağlantı koptuğunda (keep-alive süresi dolduğunda) broker tarafından.

K3.2 · ESP-IDF'de WiFi ve MQTT istemcisi

1. MQTT bağlantısı kopukken gelen CAN verisine ne olmalı?

Sınırlı boyutlu bir halka tamponda tutulmalı ve bağlantı gelince gönderilmeli; tampon dolarsa en eski veri atılmalı ve kayıp sayısı raporlanmalı.

K3.3 · Zaman serisi veri hattı: Telegraf → InfluxDB → Grafana

1. node kimliği tag mi field mı olmalı? rms_mg?

Node kimliği tag (filtreleme ve gruplama için), rms_mg field (ölçülen sayısal değer).

K3.4 · Docker Compose

1. InfluxDB verisi konteyner silinince neden kaybolmaz?

Veri dizini bir volume'a bağlanır; volume konteynerden bağımsız olarak saklanır.

K3.5 · Watchdog tasarımı

1. Watchdog'u neden sadece tek bir yerden beslersin?

Böylece besleme kararı tüm task'ların sağlığına bağlanır; her task kendisi beslerse biri kilitlendiğinde diğerleri beslemeye devam eder ve arıza gizlenir.

K3.6 · Hata enjeksiyonu ve dayanıklılık testi

1. CAN kablosu 10 saniye çekili kalırsa STM32 ve ESP32 tarafında ne görmeyi beklersin?

İki taraf da 1,5 s içinde karşıyı offline işaretler; gönderme hataları nedeniyle TEC artar, belki bus-off olur. Kablo takılınca otomatik toparlanıp heartbeat'ler geri gelir.

Hafta 4

K4.1 · Bellek haritası ve linker script

1. Metadata'yı neden uygulamayla aynı sektöre koymuyoruz?

Flash sektör bazında silinir; metadata'yı güncellemek için uygulamanın bir kısmını da silmek gerekirdi.

K4.2 · Vector table, VTOR ve açılış sırası

1. Uygulama imajının ilk 8 baytı ne içerir?

İlk 4 bayt başlangıç yığın işaretçisi (_estack), sonraki 4 bayt Reset_Handler adresi.

2. Bootloader uygulamanın geçerli olduğunu anlamak için ilk kelimeye neden bakar?

Geçerli bir SP SRAM aralığında olmalıdır; silinmiş flash 0xFFFFFFFF okur, bu da boş veya bozuk imajı gösterir.

K4.3 · Bootloader tasarımı

1. Atlamadan önce PRIMASK'ı geri açmazsan ne olur?

Uygulama kesmeler kapalı başlar; HAL_Delay, CAN kesmeleri ve FreeRTOS zamanlayıcısı çalışmaz, sistem donmuş gibi görünür.

K4.4 · Flash programlama

1. Silmeden bir kelimeyi 0x00 yazıp sonra 0xFF yazmaya çalışırsan ne olur?

0'dan 1'e yazma yapılamaz; ikinci yazma hata verir ya da değer değişmez. Önce sektör silinmelidir.

K4.5 · CRC

1. CRC32 ile imzalamak aynı şey mi?

Hayır. CRC kazara bozulmayı yakalar; kötü niyetli biri veriyi değiştirip CRC'yi yeniden hesaplayabilir. Kimlik doğrulama için kriptografik imza gerekir.

K4.6 · Güvenilir güncelleme mimarisi

1. Güncelleme sırasında son DATA paketinden hemen sonra güç kesilirse ne olur?

END gelmediği için CRC doğrulanmamış ve metadata yazılmamıştır; kart açılışta bootloader'da kalır ve güncelleme tekrarlanabilir.

K4.7 · ESP32 bölüm tablosu ve HTTP sunucusu

1. Dosyayı RAM yerine neden flash bölümüne yazıyoruz?

ESP32'nin RAM'i birkaç yüz KB ile sınırlıdır ve güncelleme uzun sürer; flash'a yazmak hem boyut sınırını kaldırır hem de aktarım yarıda kalırsa baştan indirmeden tekrar denemeyi sağlar.

Ek B. Kaynak Listesi

Dokümanda geçen tüm kaynaklar; hepsi ücretsiz erişilebilir. Adresler hazırlanırken kontrol edildi, ancak resmi doküman sitelerinin yapısı zamanla değişebilir. Bir bağlantı açılmazsa kaynağın başlığını aratman yeterli.

#	Kaynak	Tür	Adres
1	Pro Git (Türkçe çeviri) — Bölüm 1-3	Kitap	https://git-scm.com/book/tr/v2
2	GitHub Actions dokümantasyonu	Doküman	https://docs.github.com/en/actions
3	CSS Electronics — CAN Bus Explained: A Simple Intro	Okuma + video	https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial
4	Texas Instruments SLOA101 — Introduction to the Controller Area Network (CAN)	Uygulama notu	https://ti.com/lit/SLOA101
5	ST Community — CAN (bxCAN) bit time configuration on STM32 MCUs	Okuma	https://community.st.com/stm32-mcus-60/can-bxcn-bit-time-configuration-on-stm32-mcus-135466
6	CAN bit timing hesaplayıcı (bittiming.can-wiki.info)	Araç	http://www.bittiming.can-wiki.info/
7	CSS Electronics — CAN Bus Errors Explained	Okuma	https://www.csselectronics.com/pages/can-bus-errors-intro-tutorial
8	CSS Electronics — CAN DBC File Explained	Okuma	https://www.csselectronics.com/pages/can-dbc-file-database-intro
9	cantools dokümantasyonu	Doküman	https://cantools.readthedocs.io
10	Digi-Key — Introduction to RTOS (12 bölümlük video serisi)	Video	https://www.youtube.com/playlist?list=PLEBQazB0HUyQ4hAPU1cJED6t3DU0h34bz
11	Mastering the FreeRTOS Real Time Kernel (ücretsiz kitap)	Kitap	https://github.com/FreeRTOS/FreeRTOS-Kernel-Book
12	FreeRTOS — Running the RTOS on an ARM Cortex-M Core (kesme öncelikleri)	Doküman	https://www.freertos.org/RTOS-Cortex-M3-M4.html
13	Memfault Interrupt — A Practical Guide to ARM Cortex-M Exception Handling	Okuma	https://interrupt.memfault.com/blog/arm-cortex-m-exceptions-and-nvic
14	ST UM1472 — STM32F4DISCOVERY kullanıcı kılavuzu	Kılavuz	https://www.st.com/resource/en/user_manual/dm00039084-discovery-kit-with-stm32f407vg-mcu-stmicroelectronics.pdf
15	ST RM0090 — STM32F407 referans kılavuzu	Kılavuz	https://www.st.com/resource/en/reference_manual/dm00031020.pdf
16	ESP-IDF — TWAI (CAN) sürücü dokümantasyonu	Doküman	https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/peripherals/twai.html
17	Atadiat — Saleae klonu 24 MHz 8 kanal lojik analizör uygulamalı test	Okuma	https://atadiat.com/en/e-hands-on-test-saleae-usb-logic-analyzer-24mhz-8ch-clone/
18	sigrok / PulseView resmi sitesi	Araç	https://sigrok.org
19	SparkFun — Serial Peripheral Interface (SPI)	Okuma	https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi
20	ST — LIS3DSH ürün sayfası ve datasheet	Datasheet	https://www.st.com/en/mems-and-sensors/lis3dsh.html
21	Steven W. Smith — The Scientist and Engineer's Guide to DSP (ücretsiz kitap)	Kitap	http://www.dspguide.com/
22	3Blue1Brown — But what is the Fourier Transform? A visual introduction	Video	https://www.youtube.com/watch?v=spUNpyF58BY
23	National Instruments — Understanding FFTs and Windowing	Uygulama notu	https://download.ni.com/evaluation/pxi/Understanding%20FFTs%20and%20Windowing.pdf
24	CMSIS-DSP — Real FFT F32 fonksiyonları	Doküman	https://arm-software.github.io/CMSIS-DSP/latest/group__RealFFT_F32.html
25	Valmet — An introduction to troubleshooting with vibration signatures	Okuma	https://www.valmet.com/insights/articles/up-and-running/reliability/VibrationSignatures/
26	Fabrics — Vibration spectrum analysis (arıza-frekans tablosu)	Okuma	https://www.fabrics.io/blog/vibration-spectrum-analysis/

#	Kaynak	Tür	Adres
27	NIST/SEMATECH e-Handbook — Detection of Outliers	Okuma	https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm
28	ST AN4776 — How to use general-purpose timer peripheral on STM32 MCUs	Uygulama notu	https://www.st.com.cn/resource/en/application_note/an4776-how-to-use-generalpurpose-timer-peripheral-on-stm32-mcus-stmicroelectronics.pdf
29	Noctua — PWM specifications white paper (4 pinli fan standardı)	Teknik doküman	https://noctua.at/pub/media/wysiwyg/Noctua_PWM_specifications_white_paper.pdf
30	SparkFun — Pulse Width Modulation	Okuma	https://learn.sparkfun.com/tutorials/pulse-width-modulation
31	ThrowTheSwitch — Unity (C için unit test çatısı)	Doküman	https://throwtheswitch.org/unity
32	HiveMQ — MQTT Essentials serisi (tüm bölümlerin listesi)	Okuma	https://www.hivemq.com/mqtt-essentials-wrap-up/
33	HiveMQ — MQTT Essentials Part 8: Retained Messages	Okuma	https://www.hivemq.com/blog/mqtt-essentials-part-8-retained-messages/
34	ESP-IDF — ESP-MQTT dokümantasyonu	Doküman	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html
35	InfluxData — Telegraf ile MQTT'den veri toplama örneği	Doküman	https://docs.influxdata.com/telegraf/v1/examples/collect-mqtt/
36	Grafana — Provisioning dokümantasyonu	Doküman	https://grafana.com/docs/grafana/latest/administration/provisioning/
37	Docker Compose dokümantasyonu	Doküman	https://docs.docker.com/compose/
38	Mosquitto — mosquitto.conf başvuru sayfası	Doküman	https://mosquitto.org/man/mosquitto-conf-5.html
39	Memfault Interrupt — From Zero to main(): Demystifying Firmware Linker Scripts	Okuma	https://interrupt.memfault.com/blog/how-to-write-linker-scripts-for-firmware
40	Memfault Interrupt — How to Write a Bootloader from Scratch	Okuma	https://interrupt.memfault.com/blog/how-to-write-a-bootloader-from-scratch
41	Ross Williams — A Painless Guide to CRC Error Detection Algorithms	Okuma	http://www.ross.net/crc/
42	Memfault Interrupt — Device Firmware Update Cookbook	Okuma	https://interrupt.memfault.com/blog/device-firmware-update-cookbook
43	Memfault Interrupt — Secure Firmware Updates with Code Signing	Okuma	https://interrupt.memfault.com/blog/secure-firmware-updates-with-code-signing
44	ESP-IDF — Partition Tables	Doküman	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/partition-tables.html
45	ESP-IDF — HTTP Server	Doküman	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_server.html